

République Tunisienne

Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

ISSET

**Institut Supérieur des Études
Technologiques de Nabeul**

Département Technologies de l'Informatique

RAPPORT DE PROJET DE FIN D'ÉTUDES

Pour l'obtention du diplôme de
**Licence Appliquée en Technologies de
l'Informatique**

Mise en place d'un Robot de Tests

**Automatisés pour les Projets
Speed Line & Delivery App**

Réalisé par : [Nom Stagiaire 1]

: [Nom Stagiaire 2]

Encadrant entreprise : M. Mahmoud Slama

Encadrant académique : [Nom Encadrant ISET]

Entreprise d'accueil : DevWise – Nabeul, Tunisie

Année universitaire 2025 – 2026

Dédicaces

À nos chers parents,

À ceux qui nous ont donné la vie, qui ont sacrifié leur jeunesse pour notre bonheur et notre réussite, à nos chers parents. Aucune dédicace ne saurait exprimer notre respect, notre amour éternel et notre reconnaissance pour les sacrifices que vous avez consentis pour notre instruction et notre bien-être.

À nos frères et sœurs, pour leur soutien moral et leur encouragement permanent durant toutes ces années d'études.

À nos amis et collègues, pour tous les moments partagés ensemble, les encouragements et le soutien tout au long de ce parcours.

À tous ceux qui nous sont chers et que nous n'avons pas cités, nous leur dédions ce modeste travail.

[Nom Stagiaire 1] & [Nom Stagiaire 2]

Remerciements

Au terme de ce projet de fin d'études, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Nous adressons nos sincères remerciements à **M. Mahmoud Slama**, notre encadrant au sein de l'entreprise **DevWise**, pour sa disponibilité, ses conseils avisés, son suivi rigoureux et son accompagnement tout au long de ce stage. Sa confiance et son expertise nous ont permis de mener à bien ce projet dans les meilleures conditions.

Nous tenons également à remercier [**Nom Encadrant Académique**], notre encadrant académique à l'**ISSET de Nabeul**, pour son orientation, ses remarques constructives et le temps qu'il nous a consacré malgré ses nombreuses obligations.

Nos remerciements vont également à toute l'équipe de **DevWise** pour leur accueil chaleureux, leur esprit de collaboration et le cadre de travail agréable qu'ils nous ont offert durant ces quatre mois de stage. Leur professionnalisme et leur bienveillance ont grandement facilité notre intégration et notre apprentissage.

Nous exprimons notre reconnaissance à l'ensemble du corps enseignant du **Département Technologies de l'Informatique** de l'ISSET de Nabeul, pour la formation de qualité qu'ils nous ont dispensée tout au long de notre cursus universitaire. Les connaissances acquises ont été essentielles à la réalisation de ce projet.

Nous remercions les membres du jury d'avoir accepté d'évaluer notre travail. Leurs observations et recommandations ne manqueront pas d'enrichir ce rapport.

Enfin, nous adressons nos plus profonds remerciements à nos familles pour leur soutien inconditionnel, leur patience et leurs encouragements permanents. Sans leur présence et leur dévouement, ce travail n'aurait pas vu le jour.

À tous ceux qui, de près ou de loin, ont contribué à l'aboutissement de ce projet, nous

adressons nos sincères remerciements.

Table des matières

Dédicaces	i
Remerciements	ii
Introduction générale	1
1 Présentation Générale et Étude de l’Existant	5
1.1 Présentation de l’entreprise d’accueil	5
1.1.1 Présentation générale	5
1.1.2 Fiche signalétique	5
1.1.3 Structure organisationnelle	6
1.1.4 Valeurs et mission	6
1.2 Présentation des projets	7
1.2.1 Speed Line	7
1.2.2 Delivery App	8
1.3 Architecture technique existante	9
1.3.1 Inventaire des microservices	10
1.3.2 Composants d’infrastructure	10
1.3.3 Technologies d’infrastructure	11
1.3.4 Pipeline CI/CD	11
1.3.5 Diagramme d’architecture	12
1.4 Étude de l’existant : état des tests	12
1.4.1 Couverture de tests avant le stage	12
1.4.2 Types de tests absents	13
1.4.3 Conséquences observées	13
1.5 Problématique	14
1.6 Solution proposée	15
1.6.1 Pyramide de tests	15
1.6.2 Axes de la solution	15
1.6.3 Tests de performance	16
1.6.4 Intégration CI/CD	16
1.6.5 Rapports visuels	16
1.7 Méthodologie de travail	16
1.7.1 Approche Agile / Scrum	16

1.7.2	Planning prévisionnel	17
1.7.3	Outils de travail	18
2	Analyse et Conception	20
2.1	Analyse des besoins	20
2.1.1	Besoins fonctionnels	20
2.1.2	Besoins non fonctionnels	23
2.2	Diagrammes UML	25
2.2.1	Diagramme de cas d'utilisation	25
2.2.2	Diagramme de séquence – Exécution des tests	27
2.2.3	Diagramme de classes – Architecture du Robot	29
2.2.4	Diagramme de déploiement	30
2.3	Conception de la pyramide de tests	31
2.3.1	Couche 1 – Tests unitaires	31
2.3.2	Couche 2 – Tests d'intégration et API	32
2.3.3	Couche 3 – Tests End-to-End (E2E)	32
2.3.4	Couche 4 – Tests de performance	32
2.3.5	Tests de sécurité – Couche transversale	33
2.4	Plan de test global	33
2.5	Choix techniques	35
2.5.1	Playwright vs Cypress	35
2.5.2	Patrol vs Appium	37
2.5.3	k6 vs JMeter vs Gatling	38
2.5.4	Allure vs HTML Reporter	39
2.6	Architecture du robot de test	40
2.6.1	Structure des répertoires	40
2.6.2	Architecture logique	41
2.6.3	Description des modules	41
3	Réalisation	44
3.1	Environnement de développement	44
3.2	Mise en place des tests unitaires (Couche 1)	46
3.2.1	Tests unitaires backend (JUnit 5 + Mockito)	46
3.2.2	Tests unitaires frontend (Jasmine + Karma)	50
3.3	Mise en place des tests API (Couche 2)	54
3.3.1	Architecture des tests API	54
3.3.2	Tests par service	55
3.4	Mise en place des tests E2E (Couche 3)	59
3.4.1	Tests Web (Playwright)	60
3.4.2	Tests Mobile (Flutter + Patrol)	63

3.5	Mise en place des tests de performance (Couche 4)	66
3.5.1	Test de charge (<i>Load Test</i>)	66
3.5.2	Test de stress (<i>Stress Test</i>)	68
3.5.3	Test de pic (<i>Spike Test</i>)	68
3.6	Tests de sécurité	69
3.6.1	Injection SQL	69
3.6.2	Cross-Site Scripting (XSS)	70
3.6.3	Contournement d'authentification et contrôle d'accès	70
3.6.4	Limitation de débit et validation des entrées	71
3.7	Seed de la base de données	72
3.7.1	Approche technique	72
3.7.2	Comptes de test	73
3.8	Intégration CI/CD	74
3.8.1	Architecture du pipeline de tests	74
3.8.2	Stratégie de déclenchement	76
3.8.3	Gestion des dépendances entre jobs	76
3.9	Reporting avec Allure	77
3.9.1	Configuration dans Playwright	77
3.9.2	Fonctionnalités du rapport Allure	78
4	Tests et Validation	80
4.1	Environnement de test	80
4.1.1	Environnement local	80
4.1.2	Environnement d'intégration continue	81
4.2	Résultats des tests unitaires	82
4.2.1	Résultats par microservice	82
4.2.2	Analyse de la couverture par service	82
4.2.3	Types de tests unitaires implémentés	83
4.3	Résultats des tests frontend	83
4.3.1	Admin Panel	83
4.3.2	Partner Dashboard	84
4.3.3	Synthèse des tests frontend	84
4.4	Résultats des tests Playwright	84
4.4.1	Vue d'ensemble	85
4.4.2	Tests d'API (smoke et CRUD)	85
4.4.3	Tests end-to-end (E2E)	85
4.4.4	Tests de contrat et WebSocket	86
4.4.5	Tests de régression visuelle et d'accessibilité	86
4.5	Résultats des tests de performance	86

4.5.1	Scénarios de test	87
4.5.2	Seuils de performance configurés	87
4.5.3	Résultats observés	87
4.6	Résultats des tests de sécurité	88
4.6.1	Catégories de tests de sécurité	88
4.6.2	Détails des tests de sécurité	88
4.7	Couverture globale du projet	89
4.7.1	Comparaison avant/après	89
4.7.2	Analyse de l'évolution	89
4.7.3	Pyramide de tests réalisée	90
4.8	Problèmes rencontrés et solutions	90
4.8.1	Problèmes liés à l'environnement	91
4.8.2	Problèmes liés aux tests Java	91
4.8.3	Problèmes liés aux tests frontend	92
4.8.4	Problèmes liés aux tests mobiles	92
4.9	Recommandations qualité	92
4.9.1	Amélioration de la couverture de code	93
4.9.2	Extension des tests de contrat	93
4.9.3	Tests de performance en pré-production	93
4.9.4	Rapports et suivi historique	93
Conclusion Générale		95
Bibliographie et Webographie		98
A Extraits de Code		100
A.1	Configuration Playwright	100
A.2	Client API réutilisable	101
A.3	Test unitaire backend — AuthServiceImpl	102
A.4	Test E2E — Page Object Model	103
A.5	Test de performance k6	104
A.6	Test de sécurité — Injection SQL	105
B Configurations		106
B.1	Variables d'environnement (.env)	106
B.2	Package.json — Dépendances du robot	106
B.3	Configuration JaCoCo (couverture de code Java)	107
B.4	Pipeline GitLab CI	108
B.5	Configuration Patrol (tests mobiles Flutter)	109
B.6	Couverture Angular (Istanbul/Karma)	109

Table des figures

1.1	Écosystème Delivery App – Interactions entre les applications	9
1.2	Architecture microservices de Speed Line et Delivery App	12
1.3	Pyramide de tests – Solution proposée	15
2.1	Diagramme de cas d'utilisation du robot de tests	26
2.2	Diagramme de séquence – Flux d'exécution des tests automatisés . . .	28
2.3	Diagramme de classes – Architecture du robot de tests	29
2.4	Diagramme de déploiement du robot de tests	30
2.5	Pyramide de tests adoptée pour le robot de tests automatisés	31
2.6	Architecture logique du robot de tests	41
4.1	Pyramide de tests réalisée pour le projet Speed Line	90

Liste des tableaux

1.1	Fiche signalétique de DevWise	6
1.2	Stack technique de Speed Line	8
1.3	Inventaire des microservices	10
1.4	Couverture de tests par microservice avant le stage	13
1.5	Planning prévisionnel des sprints	17
1.6	Outils de travail	18
2.1	Besoins fonctionnels du robot de tests automatisés	21
2.2	Besoins non fonctionnels du robot de tests	24
2.3	Plan de test global du robot de tests automatisés	33
2.4	Répartition des scénarios par type de test	35
2.5	Comparaison Playwright vs Cypress	36
2.6	Comparaison Patrol vs Appium	37
2.7	Comparaison k6 vs JMeter vs Gatling	38
2.8	Comparaison Allure Report vs HTML Reporter	39
3.1	Environnement de développement et outils utilisés	45
3.2	Répartition des tests unitaires backend par microservice	49
3.3	Résultats des tests unitaires frontend	53
3.4	Répartition des tests API par fichier de spécification	59
3.5	Comptes de test créés par le script de seed	74
4.1	Configuration de l'environnement local de test	81
4.2	Configuration de l'environnement CI/CD	81
4.3	Résultats des tests unitaires par microservice	82
4.4	Résultats des tests du panneau d'administration	83
4.5	Résultats des tests du tableau de bord partenaire	84
4.6	Synthèse des résultats des tests frontend	84
4.7	Répartition des tests Playwright par catégorie	85
4.8	Scénarios de tests de performance k6	87
4.9	Seuils de performance définis	87
4.10	Résultats des tests de sécurité par catégorie	88
4.11	Comparaison de la couverture de test avant et après le projet	89
4.12	Problèmes d'environnement et solutions appliquées	91
4.13	Problèmes de tests Java et solutions appliquées	91

4.14 Problèmes de tests frontend et solutions appliquées	92
4.15 Problèmes de tests mobiles et solutions appliquées	92
4.16 Comparaison avant/après la mise en place du robot de tests	96

Introduction générale

La transformation numérique constitue aujourd’hui un levier stratégique pour les entreprises de tous les secteurs. Dans le domaine de la gestion commerciale et de la livraison, cette transformation se traduit par le déploiement de solutions logicielles de plus en plus sophistiquées, capables de gérer l’ensemble de la chaîne de valeur : de la prise de commande à la facturation, en passant par le suivi en temps réel des livraisons et la gestion des stocks.

Le marché tunisien, à l’instar des marchés internationaux, connaît une croissance significative de la demande en solutions de gestion intégrées. Les entreprises cherchent à optimiser leurs processus opérationnels, à réduire les erreurs humaines et à améliorer la satisfaction de leurs clients. Cette tendance s’accompagne d’exigences croissantes en matière de fiabilité, de performance et de qualité des applications déployées.

C’est dans ce contexte que s’inscrit l’activité de **DevWise**, une entreprise basée à Nabeul spécialisée dans le développement de solutions logicielles innovantes. DevWise conçoit et maintient deux produits phares :

- **Speed Line** : une application de gestion commerciale complète permettant aux entreprises de gérer leurs ventes, leurs achats, leur stock, leur facturation et leurs tableaux de bord décisionnels. Speed Line se décline en version web et bureau, offrant une flexibilité d’utilisation adaptée aux besoins de chaque utilisateur.
- **Delivery App** : un écosystème de quatre applications (web et mobile) dédiées à la gestion du cycle complet de livraison. Cet écosystème couvre les besoins des différents acteurs : les administrateurs pour le pilotage global, les livreurs pour le suivi terrain, les clients pour le passage et le suivi de commandes, ainsi qu’une interface de gestion des restaurants et commerces partenaires.

Ces deux produits, en évolution constante, représentent un périmètre fonctionnel riche et complexe. Chaque nouvelle version apporte son lot de fonctionnalités, de corrections et d’améliorations, ce qui multiplie les scénarios à valider avant chaque mise en production.

Face à cette complexité croissante, la qualité logicielle devient un enjeu majeur. Les

tests manuels, bien qu'indispensables dans certaines phases du développement, présentent des limites significatives lorsqu'ils constituent l'unique méthode de validation :

- **Coût en temps** : l'exécution manuelle de l'ensemble des scénarios de test devient de plus en plus chronophage à mesure que l'application s'étoffe. Un cycle de test complet peut nécessiter plusieurs jours de travail.
- **Risque d'erreur humaine** : la répétition de tâches identiques favorise l'inattention et les oublis. Certains cas limites ou régressions peuvent passer inaperçus.
- **Fréquence de test limitée** : le coût des tests manuels empêche leur exécution systématique à chaque modification du code source, retardant ainsi la détection des anomalies.
- **Difficulté de mise à l'échelle** : avec la multiplication des plateformes (web, Android, iOS) et des configurations, la couverture exhaustive devient pratiquement impossible par des moyens exclusivement manuels.

L'automatisation des tests apparaît donc comme une réponse adaptée à ces défis. En mettant en place un robot de tests automatisés, il devient possible d'exécuter rapidement et de manière répétable un grand nombre de scénarios, d'identifier les régressions dès leur apparition et de libérer du temps pour les équipes de développement, qui peuvent alors se concentrer sur des tâches à plus forte valeur ajoutée.

C'est dans cette perspective que s'inscrit notre projet de fin d'études, réalisé au sein de l'entreprise DevWise sur une période de quatre mois, sous l'encadrement de M. Mahmoud Slama. Les objectifs principaux de ce stage sont les suivants :

1. **Étudier l'existant** : analyser l'architecture des projets Speed Line et Delivery App, identifier les fonctionnalités critiques et recenser les scénarios de test prioritaires.
2. **Concevoir une architecture de tests** : définir une architecture modulaire et maintenable pour le robot de tests, en sélectionnant les outils et frameworks les plus adaptés au contexte technique des projets.
3. **Implémenter les tests automatisés** : développer les scripts de tests couvrant les scénarios fonctionnels principaux des deux produits, en adoptant les bonnes pratiques du *test automation*.
4. **Intégrer les tests dans le processus de développement** : mettre en place un pipeline d'intégration continue permettant l'exécution automatique des tests à chaque modification du code.
5. **Produire des rapports de tests** : générer des rapports détaillés et exploitables pour faciliter le suivi de la qualité et la prise de décision.

Pour mener à bien ce projet, nous avons adopté une **méthodologie agile** inspirée du framework **Scrum**. Cette approche itérative et incrémentale nous a permis de :

- Découper le projet en *sprints* de deux semaines, chacun délivrant un ensemble cohérent de tests automatisés.
- Organiser des réunions régulières avec l'encadrant pour synchroniser les priorités et ajuster le plan de travail.
- Intégrer les retours d'équipe de manière continue pour améliorer la qualité des livrables.
- Maintenir un backlog priorisé des scénarios à automatiser, en fonction de leur criticité et de leur fréquence d'utilisation.

Cette méthodologie s'est avérée particulièrement adaptée au contexte du projet, où les exigences pouvaient évoluer en fonction des nouvelles versions des applications et des retours des utilisateurs.

Le présent rapport est structuré en plusieurs chapitres, organisés de manière à retracer la démarche suivie tout au long de ce stage :

- **Chapitre 1 : Présentation de l'organisme d'accueil et cadre du projet.** Ce chapitre présente l'entreprise DevWise, son domaine d'activité, les projets Speed Line et Delivery App, ainsi que le cadre général et la problématique du stage.
- **Chapitre 2 : État de l'art et étude théorique.** Nous y passons en revue les concepts fondamentaux liés aux tests logiciels, les différentes stratégies d'automatisation, ainsi qu'une étude comparative des outils et frameworks disponibles sur le marché.
- **Chapitre 3 : Analyse et spécification des besoins.** Ce chapitre détaille l'analyse des besoins fonctionnels et non fonctionnels, la définition des scénarios de test prioritaires et la modélisation UML associée.
- **Chapitre 4 : Conception et architecture de la solution.** Nous y présentons l'architecture technique retenue pour le robot de tests, les choix technologiques effectués et la conception détaillée des différents modules.
- **Chapitre 5 : Réalisation et mise en œuvre.** Ce dernier chapitre couvre l'implémentation des tests automatisés, leur intégration dans le pipeline CI/CD, les résultats obtenus et les captures d'écran illustrant le fonctionnement du robot.

Enfin, nous concluons ce rapport par un bilan général du stage, en mettant en évidence

les compétences acquises, les difficultés rencontrées et les perspectives d'évolution du projet.

Chapitre 1

Présentation Générale et Étude de l'Existant

Introduction

Ce premier chapitre constitue le socle de notre rapport de projet de fin d'études. Il vise à présenter le cadre général du stage, à savoir l'entreprise d'accueil, les projets sur lesquels nous avons travaillé, ainsi que l'architecture technique existante. Nous procéderons ensuite à une étude critique de l'existant en matière de tests logiciels, ce qui nous permettra de formuler la problématique et de proposer une solution adaptée. Enfin, nous exposerons la méthodologie de travail adoptée tout au long du stage.

1.1 Présentation de l'entreprise d'accueil

1.1.1 Présentation générale

DevWise est une startup tunisienne basée à Nabeul, spécialisée dans la transformation digitale des entreprises. Fondée avec la conviction que la technologie doit être un levier de croissance accessible à toutes les organisations, DevWise accompagne ses clients dans la conception, le développement et le déploiement de solutions numériques sur mesure.

L'entreprise se distingue par son expertise dans le développement d'applications web et mobiles, en s'appuyant sur des technologies modernes et des méthodologies agiles. DevWise intervient principalement dans les domaines du commerce électronique, de la logistique et de la gestion d'entreprise.

1.1.2 Fiche signalétique

Le tableau 1.1 résume les informations clés de l'entreprise d'accueil.

TABLE 1.1 – Fiche signalétique de DevWise

Désignation	Description
Raison sociale	DevWise
Forme juridique	Startup / SARL
Siège social	Nabeul, Tunisie
Secteur d'activité	Développement logiciel et transformation digitale
Domaines d'expertise	Applications web, applications mobiles, cloud
Technologies principales	Spring Boot, Angular, Flutter, Google Cloud

1.1.3 Structure organisationnelle

L'équipe de DevWise est organisée selon une structure horizontale favorisant la collaboration et l'agilité. Elle se compose de :

- **Direction technique** : responsable de la vision technologique, de l'architecture logicielle et des choix stratégiques en matière d'outils et de frameworks.
- **Équipe de développement back-end** : chargée de la conception et de l'implémentation des microservices, des API REST et de la logique métier côté serveur.
- **Équipe de développement front-end** : responsable du développement des interfaces utilisateur web (Angular) et mobiles (Flutter).
- **Équipe DevOps** : en charge de l'infrastructure cloud, de l'intégration continue, du déploiement continu et de la supervision des environnements de production.
- **Équipe qualité et tests** : dédiée à l'assurance qualité, aux tests fonctionnels et à la validation des livrables.

1.1.4 Valeurs et mission

DevWise s'appuie sur un ensemble de valeurs fondamentales qui guident son activité quotidienne :

- **Innovation** : adopter les technologies les plus récentes et les meilleures pratiques de l'industrie pour offrir des solutions performantes.
- **Qualité** : garantir un niveau de qualité élevé à travers des processus rigoureux de développement et de tests.
- **Agilité** : s'adapter rapidement aux besoins changeants des clients grâce à une méthodologie itérative.
- **Collaboration** : favoriser le travail d'équipe et la communication transparente entre toutes les parties prenantes.

La mission de DevWise est d'accompagner les entreprises tunisiennes et internationales dans leur transformation digitale en leur fournissant des solutions logicielles fiables, évolutives et centrées sur l'utilisateur.

1.2 Présentation des projets

Durant notre stage, nous avons travaillé sur deux projets complémentaires développés par DevWise : la plateforme **Speed Line** et l'écosystème **Delivery App**. Ces deux projets forment ensemble une solution complète de gestion commerciale et de livraison.

1.2.1 Speed Line

Speed Line est une plateforme de gestion d'entreprise conçue pour centraliser et automatiser les processus métier liés à la gestion commerciale. Elle offre un ensemble complet de fonctionnalités permettant aux entreprises de gérer efficacement leurs opérations quotidiennes.

Fonctionnalités principales

Les principales fonctionnalités de Speed Line sont les suivantes :

- **Gestion des commandes** : création, suivi et historique des commandes avec gestion des statuts en temps réel.
- **Gestion des clients** : répertoire complet des clients avec historique d'achats, préférences et informations de contact.
- **Gestion des livreurs** : affectation des livreurs, suivi de leurs performances et gestion de leur disponibilité.
- **Facturation** : génération automatique des factures, gestion des paiements et suivi comptable.
- **Tracking en temps réel** : suivi géolocalisé des livraisons avec notifications en temps réel.
- **Back-office d'administration** : tableau de bord complet pour la supervision de l'ensemble des opérations.

Stack technique

Le tableau 1.2 présente les technologies utilisées dans le développement de Speed Line.

TABLE 1.2 – Stack technique de Speed Line

Couche	Technologie	Version / Détails
Back-end	Spring Boot (Java)	Java 17, architecture microservices
Front-end (Admin)	Angular	Panel d'administration
Base de données relationnelle	PostgreSQL	Données transactionnelles
Cache	Redis	Sessions, cache applicatif
Infrastructure cloud	Google Cloud	Cloud Run, Pub/Sub
Conteneurisation	Docker	Images de déploiement
CI/CD	GitLab CI	Pipelines automatisés
Gestion de code	Git / GitLab	Versionnement et collaboration

1.2.2 Delivery App

Delivery App est un écosystème complet de livraison composé de quatre applications distinctes, chacune destinée à un acteur spécifique de la chaîne de livraison. Ensemble, ces applications couvrent l'intégralité du cycle de vie d'une commande, depuis la passation par le client jusqu'à la confirmation de la livraison.

Application Client (Flutter)

L'application client est développée en **Flutter** et s'adresse aux utilisateurs finaux. Elle leur permet de :

- Parcourir les catalogues des partenaires et consulter les menus.
- Passer des commandes en quelques étapes simples.
- Suivre en temps réel l'état de la livraison sur une carte interactive.
- Effectuer des paiements en ligne de manière sécurisée.
- Consulter l'historique des commandes et laisser des avis.

Application Partenaire (Angular)

L'application partenaire est une application web développée en **Angular**. Elle est destinée aux restaurants et commerces partenaires et offre les fonctionnalités suivantes :

- Gérer les commandes entrantes (acceptation, préparation, expédition).
- Administrer les menus, les produits et les prix.
- Suivre les statuts des commandes en temps réel.
- Consulter les statistiques de vente et les rapports de performance.

Application Livreur (Flutter)

L'application livreur est développée en **Flutter** et permet aux livreurs de :

- Recevoir les missions de livraison avec les détails de la commande.
- Consulter l'itinéraire optimal via une carte intégrée.
- Mettre à jour le statut de la livraison en temps réel.
- Confirmer la livraison auprès du client.
- Consulter l'historique des livraisons effectuées et les gains associés.

Application Admin (Angular)

L'application d'administration est une application web développée en **Angular** qui offre une vision globale de l'écosystème. Elle permet de :

- Superviser l'ensemble des commandes, des livraisons et des utilisateurs.
- Gérer les comptes des partenaires et des livreurs.
- Configurer les paramètres système (zones de livraison, tarification, promotions).
- Analyser les performances à travers des tableaux de bord et des rapports détaillés.
- Gérer le support client et les réclamations.

La figure 1.1 illustre les interactions entre les quatre applications de l'écosystème Delivery App.

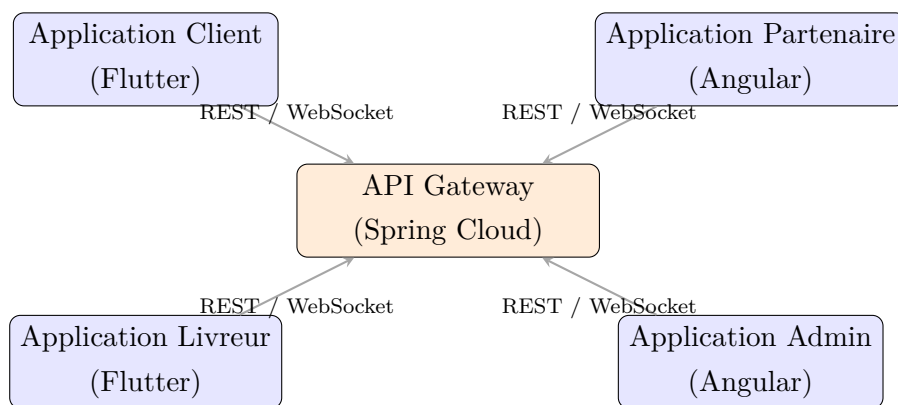


FIGURE 1.1 – Écosystème Delivery App – Interactions entre les applications

1.3 Architecture technique existante

L'architecture technique des projets Speed Line et Delivery App repose sur une approche microservices, où chaque service est responsable d'un domaine métier bien défini. Cette section détaille les différents composants de cette architecture.

1.3.1 Inventaire des microservices

Le système est composé de douze microservices, chacun encapsulant une responsabilité métier spécifique. Le tableau 1.3 présente l'inventaire complet de ces services.

TABLE 1.3 – Inventaire des microservices

Microservice	Responsabilité
auth-service	Authentification, autorisation, gestion des tokens JWT et OAuth2
user-service	Gestion des profils utilisateurs, préférences et données personnelles
partner-service	Gestion des partenaires (restaurants, commerces), leurs menus et catalogues
order-service	Cycle de vie des commandes, de la création à la clôture
delivery-service	Affectation des livreurs, gestion des tournées et suivi des livraisons
payment-service	Traitement des paiements, facturation et gestion des transactions
notification-service	Envoi de notifications push, SMS et e-mails aux différents acteurs
location-service	Géolocalisation en temps réel, calcul d'itinéraires et zones de couverture
analytics-service	Collecte et analyse des données métier, génération de rapports
promotion-service	Gestion des promotions, codes promo et offres spéciales
review-service	Gestion des avis, notes et retours des utilisateurs
support-service	Support client, gestion des tickets et des réclamations

1.3.2 Composants d'infrastructure

En plus des microservices métier, l'architecture repose sur plusieurs composants d'infrastructure essentiels :

- **API Gateway (Spring Cloud Gateway)** : point d'entrée unique pour toutes les requêtes clients. Il assure le routage, l'équilibrage de charge et la gestion des

CORS¹.

- **Eureka Server (Service Discovery)** : registre de services permettant la découverte dynamique des microservices et l'équilibrage de charge côté client.
- **Config Server (Spring Cloud Config)** : serveur de configuration centralisé permettant de gérer les propriétés de tous les services depuis un dépôt Git unique.

1.3.3 Technologies d'infrastructure

L'infrastructure de production s'appuie sur les services Google Cloud et plusieurs technologies complémentaires :

- **Google Cloud Run** : hébergement des microservices sous forme de conteneurs serverless, offrant une mise à l'échelle automatique.
- **PostgreSQL** : base de données relationnelle principale pour les données transactionnelles (commandes, utilisateurs, paiements).
- **MongoDB** : base de données NoSQL utilisée pour les données semi-structurées (logs, historiques de géolocalisation).
- **Redis** : système de cache en mémoire pour les sessions, les données fréquemment consultées et la gestion des files d'attente.
- **ClickHouse** : base de données analytique colonnes utilisée par l'`analytics-service` pour les requêtes agrégées à haute performance.
- **Google Pub/Sub** : système de messagerie asynchrone pour la communication événementielle entre microservices.

1.3.4 Pipeline CI/CD

Le déploiement est automatisé grâce à un pipeline CI/CD basé sur **GitLab CI** et **Docker**. Le processus suit les étapes suivantes :

1. **Build** : compilation du code source et construction de l'image Docker.
2. **Tests unitaires** : exécution des tests existants (avant notre intervention, cette étape était très limitée).
3. **Push de l'image** : publication de l'image Docker sur le registre Google Container Registry.
4. **Déploiement** : déploiement automatique sur Google Cloud Run.

1. Cross-Origin Resource Sharing.

1.3.5 Diagramme d'architecture

La figure 1.2 présente une vue d'ensemble de l'architecture microservices, illustrant les interactions entre les différents composants du système.

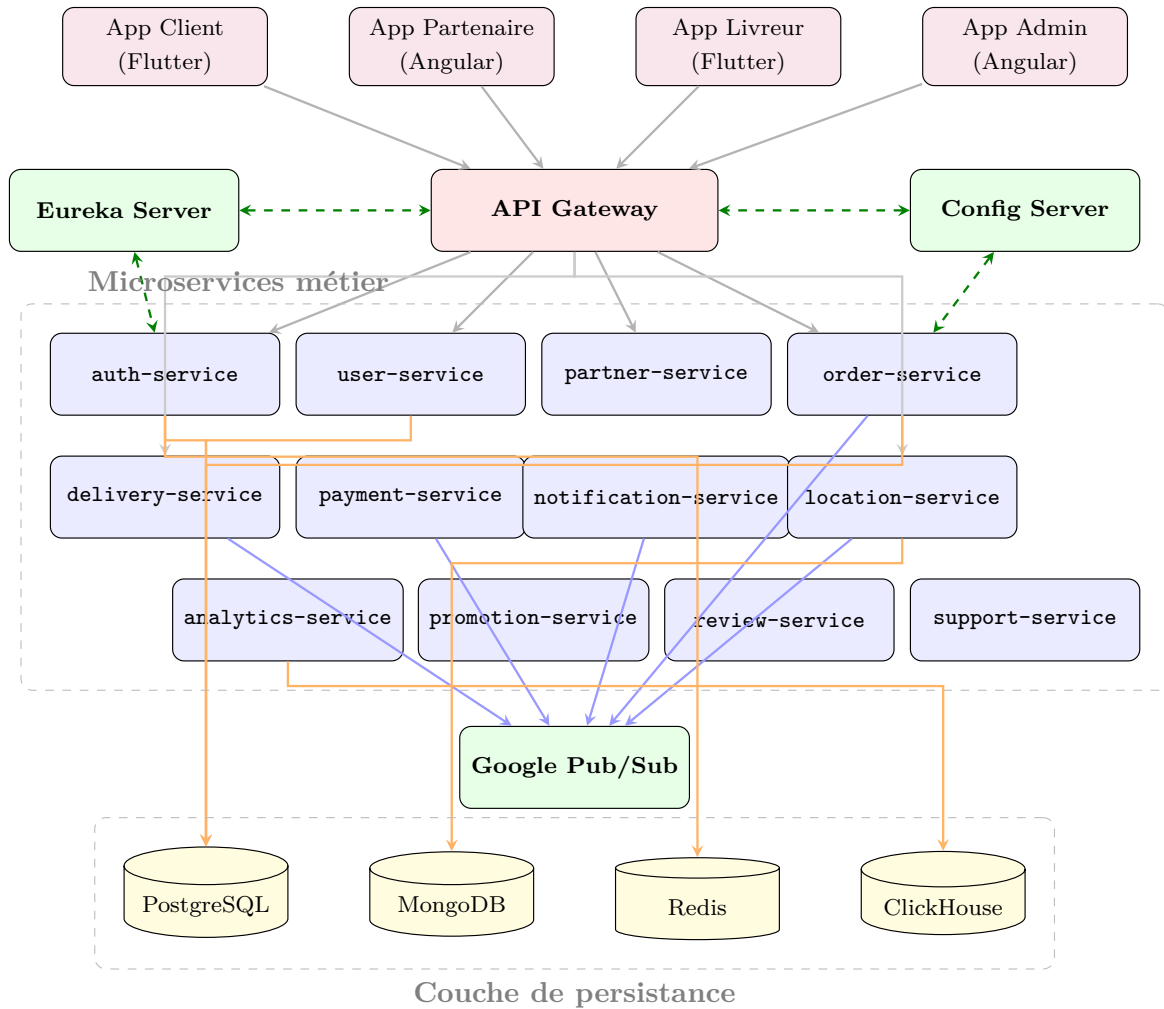


FIGURE 1.2 – Architecture microservices de Speed Line et Delivery App

1.4 Étude de l'existant : état des tests

Avant notre intervention, l'état des tests au sein des projets Speed Line et Delivery App présentait des lacunes importantes. Cette section dresse un bilan complet de la couverture de tests existante et identifie les problèmes qui en découlent.

1.4.1 Couverture de tests avant le stage

Le tableau 1.4 présente l'état de la couverture de tests pour chaque microservice avant le début du stage. On constate une disparité significative entre les services, avec une majorité de services ne disposant que d'un nombre minimal, voire nul, de fichiers de

tests.

TABLE 1.4 – Couverture de tests par microservice avant le stage

Microservice	Fichiers de tests	Couverture estimée	Observation
auth-service	2	< 5%	Tests basiques uniquement
user-service	1	< 3%	Quasi inexistant
partner-service	1	< 3%	Quasi inexistant
order-service	2	< 5%	Tests minimaux
delivery-service	41	~ 35%	Le plus couvert
payment-service	0	0%	Aucun test
notification-service	0	0%	Aucun test
location-service	1	< 3%	Quasi inexistant
analytics-service	0	0%	Aucun test
promotion-service	20	~ 20%	Partiellement couvert
review-service	0	0%	Aucun test
support-service	0	0%	Aucun test
Total	68	~ 5% (moyenne)	Très insuffisant

1.4.2 Types de tests absents

Au-delà du faible nombre de tests unitaires, plusieurs catégories de tests étaient totalement absentes :

- **Tests de bout en bout (E2E)** : aucun scénario de test simulant le parcours complet d'un utilisateur n'était en place.
- **Tests d'API** : aucune suite de tests automatisés ne validait le comportement des endpoints REST des microservices.
- **Tests de performance** : aucun test de charge ou de stress n'avait été réalisé pour évaluer la tenue en charge du système.
- **Tests mobiles** : les applications Flutter (client et livreur) ne disposaient d'aucun test automatisé, ni unitaire ni d'intégration.
- **Tests d'intégration** : les interactions entre microservices n'étaient pas testées de manière automatisée.

1.4.3 Conséquences observées

L'absence quasi totale de tests automatisés engendrait plusieurs problèmes opérationnels majeurs :

1. **Bugs en production** : des anomalies étaient régulièrement détectées par les utilisateurs finaux en environnement de production, ce qui nuisait à l'expérience utilisateur et à la crédibilité du produit.
2. **Tests manuels chronophages** : chaque nouvelle fonctionnalité ou correction de bug nécessitait une phase de test manuelle longue et fastidieuse, ralentissant considérablement le cycle de livraison.
3. **Absence de détection de régressions** : sans tests automatisés, il était impossible de détecter automatiquement les régressions introduites par de nouvelles modifications du code.
4. **Confiance réduite dans les déploiements** : l'équipe de développement manquait de confiance lors des mises en production, ce qui entraînait soit des retards, soit des déploiements risqués.
5. **Dette technique croissante** : l'accumulation de code non testé rendait la maintenance et l'évolution du système de plus en plus difficiles.

1.5 Problématique

À la lumière de l'étude de l'existant, nous pouvons formuler la problématique suivante :

Comment assurer la qualité logicielle et la fiabilité d'un écosystème distribué composé de douze microservices et de quatre applications front-end, dans un contexte de développement rapide et de déploiements fréquents, tout en réduisant les risques de régression et en accélérant le cycle de livraison ?

Cette problématique se décline en plusieurs défis concrets :

- **Complexité architecturale** : le système repose sur douze microservices interconnectés, chacun avec sa propre base de données et sa propre logique métier. La surface de test est donc considérable.
- **Diversité technologique** : l'écosystème comprend deux applications Angular et deux applications Flutter, nécessitant des outils et des approches de test différents.
- **Tests manuels inadaptés** : avec la croissance du code source et la fréquence des déploiements, les tests manuels deviennent trop lents et sujets aux erreurs humaines.
- **Absence d'intégration CI/CD** : les tests existants ne sont pas intégrés dans le pipeline de déploiement, ce qui signifie que du code non testé peut être déployé en production.

- **Risque de régression** : chaque modification du code dans un service peut avoir des répercussions sur d'autres services, et sans tests automatisés, ces régressions passent inaperçues.

1.6 Solution proposée

Pour répondre à la problématique identifiée, nous proposons la mise en place d'un **robot de tests automatisés** couvrant l'ensemble de la pyramide de tests. Cette solution s'articule autour de quatre axes principaux correspondant aux quatre niveaux de la pyramide de tests logiciels.

1.6.1 Pyramide de tests

Notre solution couvre les quatre couches de la pyramide de tests, comme illustré dans la figure 2.5 :

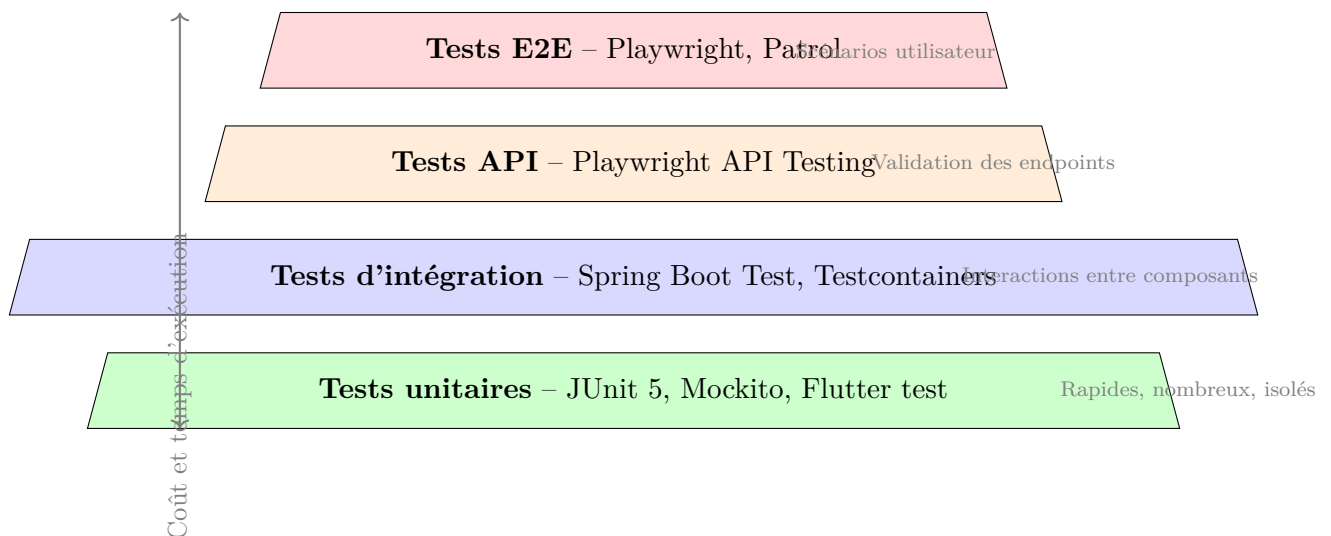


FIGURE 1.3 – Pyramide de tests – Solution proposée

1.6.2 Axes de la solution

1. **Tests unitaires (couche de base)** : écriture de tests unitaires pour chaque microservice à l'aide de JUnit 5 et Mockito. Ces tests valident le comportement isolé de chaque composant (services, contrôleurs, utilitaires). Pour les applications Flutter, nous utiliserons le framework de test natif.
2. **Tests d'intégration** : mise en place de tests d'intégration avec Spring Boot Test et Testcontainers pour valider les interactions entre les composants au sein d'un même service (base de données, cache, messaging).

3. **Tests API** : développement d'une suite complète de tests API avec Playwright pour valider le comportement des endpoints REST de chaque microservice.
4. **Tests de bout en bout (E2E)** : automatisation des scénarios utilisateur complets avec Playwright pour les applications web (Angular) et Patrol pour les applications mobiles (Flutter).

1.6.3 Tests de performance

En complément de la pyramide de tests, nous intégrerons des **tests de performance** à l'aide de l'outil **k6**. Ces tests permettront de :

- Mesurer les temps de réponse des API sous charge.
- Identifier les goulots d'étranglement.
- Valider la capacité du système à supporter un volume d'utilisateurs simultanés conforme aux exigences.

1.6.4 Intégration CI/CD

L'ensemble des tests sera intégré dans le pipeline CI/CD de GitLab CI. Chaque commit et chaque merge request déclenchera automatiquement l'exécution des tests pertinents, empêchant le déploiement de code non conforme.

1.6.5 Rapports visuels

Nous mettrons en place des rapports de tests visuels à l'aide d'**Allure Report**, offrant :

- Un tableau de bord interactif avec le taux de réussite global.
- Le détail de chaque test (durée, résultat, captures d'écran en cas d'échec).
- L'historique des exécutions pour suivre l'évolution de la couverture.
- Des graphiques de tendance pour identifier les tests instables.

1.7 Méthodologie de travail

1.7.1 Approche Agile / Scrum

Pour mener à bien ce projet, nous avons adopté la méthodologie **Agile Scrum**, une approche itérative et incrémentale particulièrement adaptée aux projets de développement logiciel. Cette méthodologie nous permet de :

- Livrer des incréments fonctionnels à intervalles réguliers.

- S'adapter aux changements de priorités et aux retours des parties prenantes.
- Maintenir une communication constante avec l'équipe de développement.
- Mesurer la progression du projet de manière continue.

Les cérémonies Scrum suivantes ont été mises en place :

- **Sprint Planning** : planification des tâches au début de chaque sprint (durée de 2 semaines).
- **Daily Standup** : réunion quotidienne de synchronisation avec l'équipe.
- **Sprint Review** : démonstration des résultats à la fin de chaque sprint.
- **Sprint Retrospective** : analyse de ce qui a bien fonctionné et identification des axes d'amélioration.

1.7.2 Planning prévisionnel

Le stage s'étend sur une durée de quatre mois, organisés en huit sprints de deux semaines chacun. Le tableau 1.5 présente le planning prévisionnel.

TABLE 1.5 – Planning prévisionnel des sprints

Sprint	Période	Objectifs
1	Semaines 1–2	Prise en main du projet, étude de l'existant, mise en place de l'environnement de tests
2	Semaines 3–4	Tests unitaires – <code>auth-service</code> , <code>user-service</code> , <code>partner-service</code>
3	Semaines 5–6	Tests unitaires – <code>order-service</code> , <code>delivery-service</code> , <code>payment-service</code>
4	Semaines 7–8	Tests unitaires – services restants + tests d'intégration
5	Semaines 9–10	Tests API avec Playwright pour les services critiques
6	Semaines 11–12	Tests E2E – applications Angular (Admin, Partenaire)
7	Semaines 13–14	Tests E2E – applications Flutter (Client, Livreur) + tests de performance (k6)
8	Semaines 15–16	Intégration CI/CD, rapports Allure, documentation et finalisation

1.7.3 Outils de travail

Le tableau 1.6 récapitule les principaux outils utilisés au cours du stage.

TABLE 1.6 – Outils de travail

Catégorie	Outil	Utilisation
Gestion de code	Git / GitLab	Versionnement, branches, merge requests
Tests unitaires (Java)	JUnit 5, Mockito	Tests des microservices Spring Boot
Tests unitaires (Flutter)	Flutter Test, Patrol	Tests des applications mobiles
Tests d'intégration	Spring Boot Test	Tests avec contexte applicatif
Tests d'intégration	Testcontainers	Bases de données conteneurisées pour les tests
Tests API	Playwright	Validation automatisée des endpoints REST
Tests E2E (Web)	Playwright	Automatisation des navigateurs web
Tests E2E (Mobile)	Patrol	Automatisation des applications Flutter
Tests de performance	k6	Tests de charge et de stress
Rapports	Allure Report	Génération de rapports visuels interactifs
CI/CD	GitLab CI	Automatisation du pipeline de tests
Conteneurisation	Docker	Environnements de tests isolés

Conclusion

Dans ce premier chapitre, nous avons présenté l'entreprise d'accueil DevWise ainsi que les deux projets sur lesquels porte notre stage : Speed Line et Delivery App. L'étude de l'architecture technique existante a révélé un écosystème riche composé de douze microservices et de quatre applications front-end. L'analyse de l'état des tests a mis en lumière des lacunes importantes, avec une couverture globale estimée à seulement 5%, l'absence totale de tests E2E, de tests API et de tests de performance, et aucune intégration des tests dans le pipeline CI/CD.

Face à cette problématique, nous avons proposé une solution basée sur la pyramide de tests, couvrant les tests unitaires, les tests d'intégration, les tests API et les tests E2E,

le tout intégré dans le pipeline CI/CD avec des rapports visuels Allure. La méthodologie Agile Scrum adoptée nous permettra de mener à bien ce projet de manière itérative sur une durée de quatre mois.

Le chapitre suivant sera consacré à l'état de l'art, où nous étudierons les concepts fondamentaux des tests logiciels et les outils que nous utiliserons.

Chapitre 2

Analyse et Conception

Introduction

Dans ce chapitre, nous présentons l'analyse détaillée des besoins fonctionnels et non fonctionnels de notre robot de tests automatisés, suivie de la conception architecturale à travers des diagrammes UML. Nous justifions également les choix techniques retenus et décrivons la stratégie globale de test adoptée pour les projets Speed Line et Delivery App.

2.1 Analyse des besoins

L'analyse des besoins constitue une étape fondamentale dans tout projet logiciel. Elle permet de définir avec précision les fonctionnalités attendues du système ainsi que les contraintes qualitatives auxquelles il doit répondre. Dans le cadre de notre projet, nous distinguons les besoins fonctionnels, qui décrivent les capacités du robot de tests, et les besoins non fonctionnels, qui spécifient les critères de qualité.

2.1.1 Besoins fonctionnels

Les besoins fonctionnels définissent l'ensemble des fonctionnalités que le robot de tests automatisés doit offrir. Ils ont été identifiés à partir de l'analyse des modules applicatifs des projets Speed Line et Delivery App, ainsi que des scénarios critiques de l'activité métier.

Le tableau 2.1 récapitule l'ensemble des besoins fonctionnels identifiés.

TABLE 2.1 – Besoins fonctionnels du robot de tests automatisés

ID	Module	Description
BF01	Authentification	Automatiser les tests de connexion, d'inscription et de vérification OTP pour les différentes interfaces (client, partenaire, livreur, administrateur).
BF02	Cycle de commande	Automatiser les tests du cycle complet de commande : création, acceptation par le partenaire, préparation, affectation au livreur et livraison.
BF03	Paieement	Automatiser les tests des différents modes de paiement (en ligne, à la livraison) et vérifier la cohérence des montants calculés.
BF04	Affectation livreur	Automatiser les tests d'affectation des livreurs aux commandes et le suivi en temps réel (tracking GPS).
BF05	Gestion partenaire	Automatiser les tests de gestion des menus, des catégories, de la disponibilité des produits et du traitement des commandes côté partenaire.
BF06	Gestion utilisateurs	Automatiser les tests de gestion des comptes utilisateurs : création, modification, désactivation et gestion des rôles.
BF07	Promotions	Automatiser les tests de création, d'application et de validation des codes promotionnels et des offres spéciales.
BF08	Avis et évaluations	Automatiser les tests de soumission et d'affichage des avis clients sur les partenaires et les livreurs.
BF09	Support client	Automatiser les tests du module de support : création de tickets, réponses et clôture des réclamations.
BF10	Rapports visuels	Générer des rapports de tests visuels et interactifs à l'aide d'Allure Report, incluant les captures d'écran, les traces et les statistiques d'exécution.
BF11	Intégration CI/CD	Intégrer l'exécution automatique des tests dans le pipeline CI/CD GitLab, avec déclenchement sur chaque <i>push</i> et <i>merge request</i> .

Détail des besoins fonctionnels

BF01 – Automatisation des tests d’authentification. Le module d’authentification est le point d’entrée de toutes les applications. Le robot doit vérifier les scénarios suivants :

- Connexion avec des identifiants valides et invalides.
- Inscription d’un nouveau compte avec validation des champs obligatoires.
- Envoi et vérification du code OTP par SMS.
- Gestion de la session (expiration du token, rafraîchissement).
- Réinitialisation du mot de passe.

BF02 – Automatisation des tests du cycle de commande. Le cycle de commande représente le flux métier principal de la plateforme Speed Line. Le robot doit couvrir l’intégralité du parcours :

- Ajout de produits au panier depuis l’application client.
- Validation de la commande avec choix de l’adresse de livraison.
- Réception et acceptation de la commande par le partenaire.
- Changement de statut : en préparation, prête, en livraison, livrée.
- Vérification de la cohérence des notifications à chaque étape.

BF03 – Automatisation des tests de paiement. Les tests de paiement doivent couvrir :

- Le paiement en ligne via passerelle sécurisée.
- Le paiement à la livraison.
- La vérification du calcul des montants (sous-total, frais de livraison, remises, total).
- La gestion des échecs de paiement et des remboursements.

BF04 – Automatisation des tests d’affectation livreur et tracking. Ce module critique pour l’expérience utilisateur doit être testé pour :

- L’affectation automatique et manuelle des livreurs.
- La mise à jour en temps réel de la position du livreur.
- Le calcul des estimations de temps de livraison.
- La gestion des cas d’indisponibilité des livreurs.

BF05 – Automatisation des tests de gestion partenaire. L’interface partenaire doit être testée pour :

- La gestion du menu (ajout, modification, suppression de produits).
- La gestion des catégories et des prix.
- Le basculement de la disponibilité des produits.
- Le traitement et le suivi des commandes entrantes.

2.1.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité que le robot de tests doit respecter. Ils sont essentiels pour garantir l'efficacité et la pérennité de la solution mise en place.

TABLE 2.2 – Besoins non fonctionnels du robot de tests

ID	Critère	Description	Métrique
BNF01	Performance	L'ensemble de la suite de tests doit s'exécuter en moins de 10 minutes afin de ne pas ralentir le pipeline CI/CD.	< 10 min
BNF02	Fiabilité	Les tests doivent être déterministes : un même test exécuté plusieurs fois dans les mêmes conditions doit produire le même résultat. L'objectif est de minimiser les <i>flaky tests</i> .	Taux de fiabilité > 98%
BNF03	Maintenabilité	L'architecture du robot doit suivre le patron <i>Page Object Model</i> (POM) pour séparer la logique de test de la description des pages, facilitant ainsi la maintenance.	Couplage faible
BNF04	Portabilité	Les tests doivent être exécutables aussi bien en environnement local (machine du développeur) qu'en environnement CI/CD (conteneurs Docker dans GitLab).	2 environnements
BNF05	Sécurité	Le robot doit inclure des tests de sécurité de base : détection des vulnérabilités d'injection SQL, de <i>Cross-Site Scripting</i> (XSS) et de contournement d'authentification.	Tests OWASP Top 10
BNF06	Extensibilité	Le robot doit être facilement extensible pour accueillir de nouveaux scénarios de test sans modification majeure de l'architecture existante.	Ajout en < 1 h
BNF07	Traçabilité	Chaque exécution de test doit générer un rapport détaillé avec historique, permettant de suivre l'évolution de la qualité	Rapport Allure

Performance. La contrainte de temps d'exécution inférieur à 10 minutes est dictée par les bonnes pratiques DevOps. Un pipeline trop lent décourage les développeurs et ralentit le cycle de livraison. Pour atteindre cet objectif, nous prévoyons la parallélisation des tests et l'optimisation des temps d'attente.

Fiabilité. Les *flaky tests* (tests instables qui échouent de manière aléatoire) sont l'un des principaux problèmes des suites de tests automatisés. Pour y remédier, nous adoptons les stratégies suivantes :

- Utilisation d'un script de *seed* pour initialiser la base de données dans un état connu avant chaque exécution.
- Attentes explicites (*waitFor*) au lieu de délais fixes (*sleep*).
- Isolation des tests : chaque test est indépendant des autres.
- Mécanisme de *retry* automatique pour les tests sensibles au réseau.

Maintenabilité. Le patron *Page Object Model* permet de centraliser la description des éléments d'interface dans des classes dédiées. Ainsi, lorsqu'un sélecteur CSS ou un chemin d'API change, une seule modification suffit, au lieu de mettre à jour chaque test individuellement.

Sécurité. Bien que les tests de sécurité approfondis relèvent d'une démarche de *penetration testing* dédiée, notre robot intègre des vérifications de base couvrant les vulnérabilités les plus courantes du référentiel OWASP Top 10, notamment les injections SQL et les attaques XSS.

2.2 Diagrammes UML

La modélisation UML (*Unified Modeling Language*) permet de représenter graphiquement les différents aspects du système. Nous présentons dans cette section les diagrammes de cas d'utilisation, de séquence, de classes et de déploiement du robot de tests automatisés.

2.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (figure 2.1) présente les interactions entre les acteurs du système et les principales fonctionnalités du robot de tests.

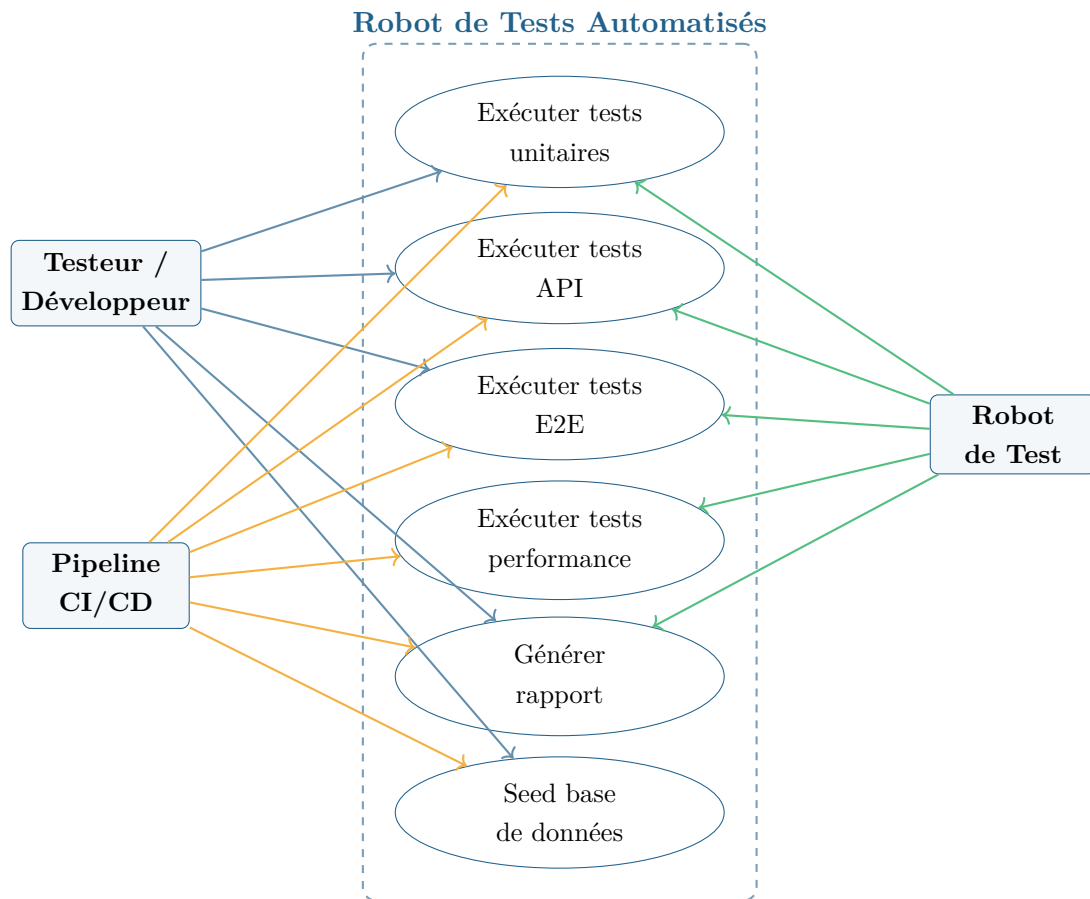


FIGURE 2.1 – Diagramme de cas d'utilisation du robot de tests

Description des acteurs.

- **Testeur / Développeur** : l'utilisateur humain qui configure, lance et analyse les résultats des tests. Il peut déclencher manuellement l'exécution de n'importe quel type de test et consulter les rapports générés.
- **Pipeline CI/CD** : le système automatisé GitLab CI qui déclenche l'exécution de la suite de tests complète à chaque événement de type *push* ou *merge request*. Il orchestre l'enchaînement des étapes de *seed*, d'exécution et de génération de rapports.
- **Robot de Test** : l'agent logiciel qui exécute effectivement les scénarios de test. Il interagit avec les applications web, les API et les applications mobiles selon les instructions codifiées dans les scripts de test.

Description des cas d'utilisation.

- **Exécuter tests unitaires** : lancer les tests unitaires JUnit 5 (backend Spring Boot), Jasmine/Karma (frontend Angular) et les tests unitaires Flutter.
- **Exécuter tests API** : lancer les tests d'intégration API via Playwright pour

vérifier les endpoints REST.

- **Exécuter tests E2E** : lancer les tests de bout en bout sur les interfaces web (Playwright) et mobile (Patrol/Flutter *integration_test*).
- **Exécuter tests performance** : lancer les scénarios de charge k6 pour évaluer les performances sous contrainte.
- **Générer rapport** : produire un rapport Allure interactif consolidant les résultats de toutes les couches de test.
- **Seed base de données** : initialiser la base de données avec un jeu de données de test cohérent et reproductible.

2.2.2 Diagramme de séquence – Exécution des tests

Le diagramme de séquence (figure 2.2) illustre le flux d'exécution complet des tests automatisés, depuis le déclenchement par le développeur jusqu'à la publication du rapport final.

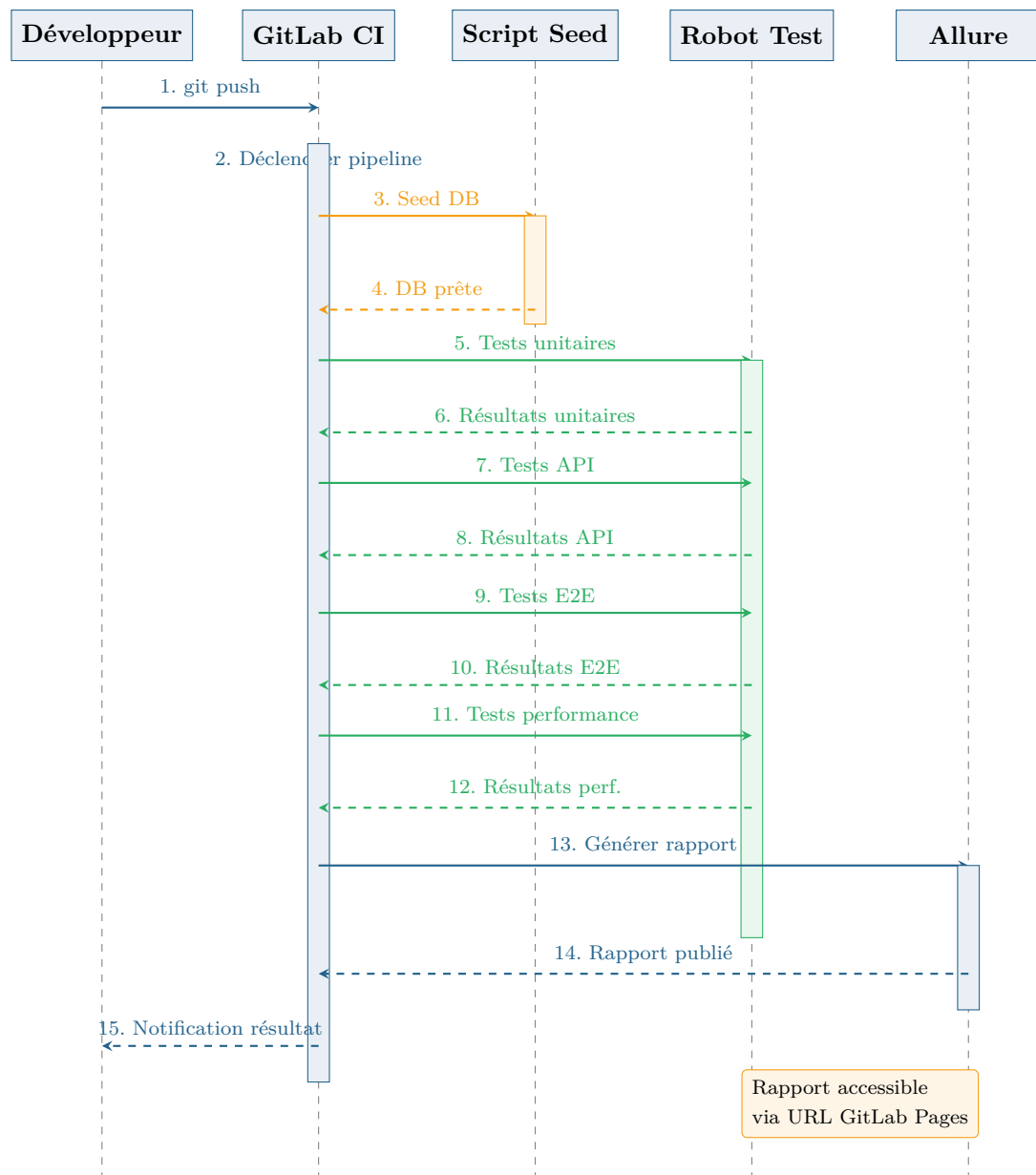


FIGURE 2.2 – Diagramme de séquence – Flux d'exécution des tests automatisés

Description du flux. Le processus d'exécution des tests suit les étapes séquentielles suivantes :

1. Le développeur pousse son code sur le dépôt GitLab (*git push*).
2. GitLab CI détecte l'événement et déclenche le pipeline de tests.
3. Le script de *seed* initialise la base de données avec un jeu de données prédéfini.
4. Une fois la base de données prête, le robot exécute successivement les tests unitaires, les tests API, les tests E2E et les tests de performance.
5. À chaque étape, les résultats sont collectés et stockés.
6. Allure génère un rapport interactif consolidant l'ensemble des résultats.
7. Le rapport est publié sur GitLab Pages et une notification est envoyée au déve-

loppieur.

2.2.3 Diagramme de classes – Architecture du Robot

Le diagramme de classes (figure 2.3) présente l’architecture orientée objet du robot de tests, mettant en évidence les principales classes et leurs relations.

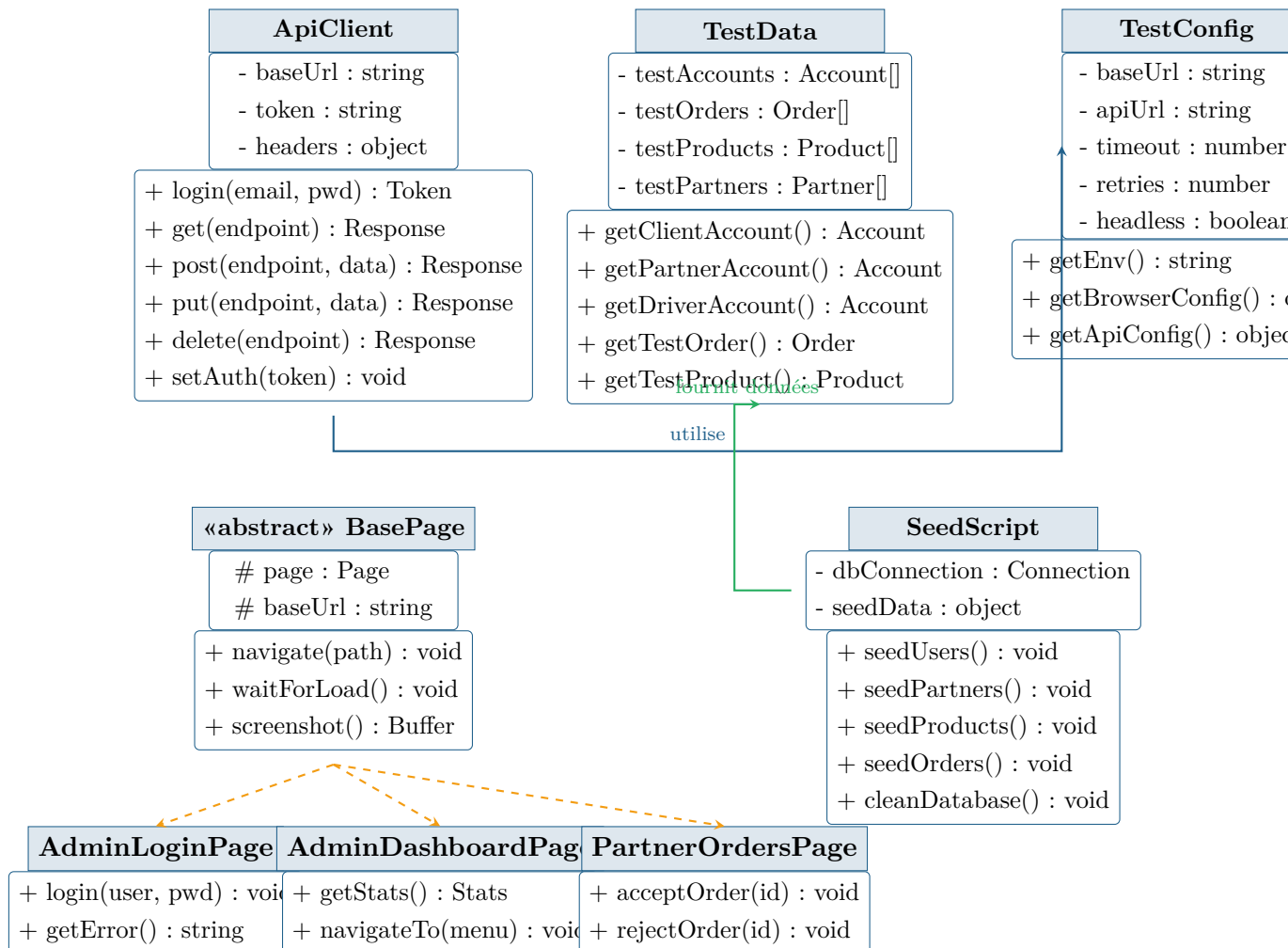


FIGURE 2.3 – Diagramme de classes – Architecture du robot de tests

Description des classes.

- **ApiClient** : classe centrale pour l’interaction avec les API REST. Elle encapsule les méthodes HTTP (GET, POST, PUT, DELETE) et gère l’authentification via un jeton JWT.
- **TestData** : classe fournissant les données de test (comptes, commandes, produits) utilisées par les différents scénarios. Elle assure la cohérence des données entre les tests.
- **TestConfig** : classe de configuration centralisant les paramètres d’environnement (URLs, timeouts, mode headless, nombre de tentatives).

- **BasePage** : classe abstraite implémentant le patron *Page Object Model*. Elle fournit les méthodes communes à toutes les pages (navigation, attente de chargement, capture d'écran).
- **AdminLoginPage**, **AdminDashboardPage**, **PartnerOrdersPage** : classes concrètes héritant de **BasePage**, chacune décrivant les éléments et les actions propres à une page spécifique.
- **SeedScript** : classe responsable de l'initialisation de la base de données avec un jeu de données de test reproductible.

2.2.4 Diagramme de déploiement

Le diagramme de déploiement (figure 2.4) décrit l'infrastructure matérielle et logicielle sur laquelle le robot de tests est déployé, ainsi que les connexions entre les différents composants.

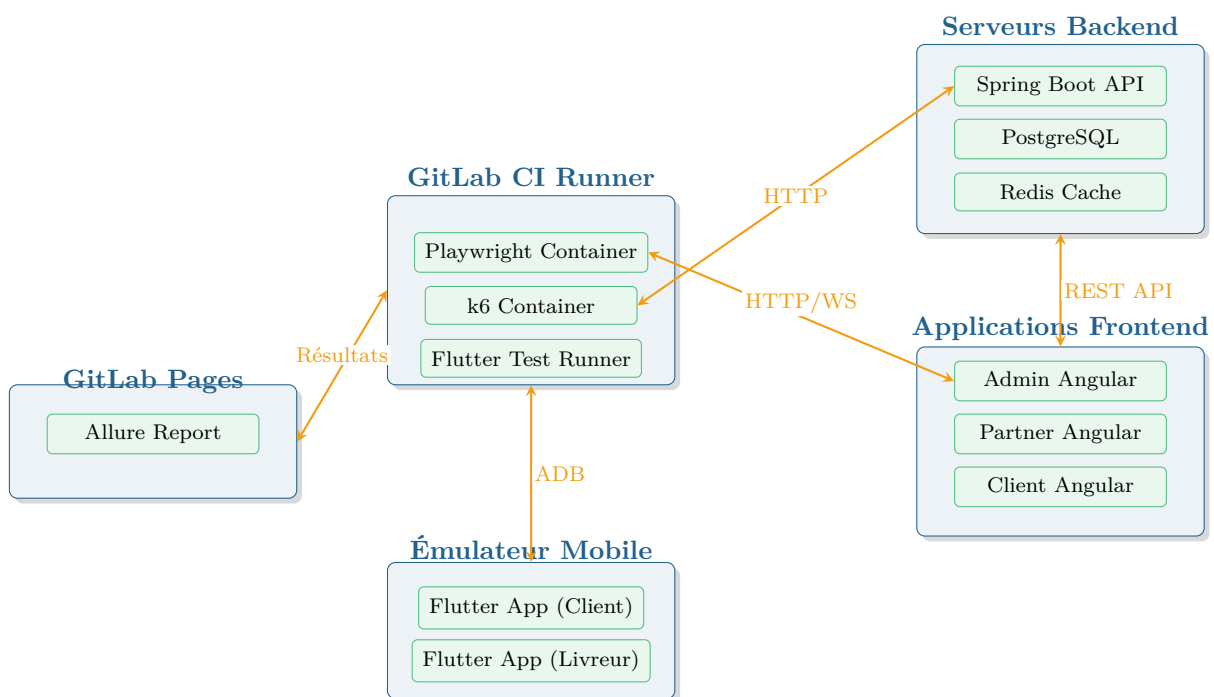


FIGURE 2.4 – Diagramme de déploiement du robot de tests

Description des nœuds.

- **GitLab CI Runner** : machine virtuelle ou conteneur Docker exécutant le pipeline CI/CD. Il héberge les trois environnements d'exécution des tests : le conteneur Playwright (tests web et API), le conteneur k6 (tests de performance) et le runner Flutter (tests mobiles).
- **Serveurs Backend** : infrastructure hébergeant l'API Spring Boot, la base de données PostgreSQL et le cache Redis. Les tests interagissent avec ces services

via des requêtes HTTP.

- **Applications Frontend** : les trois applications Angular (administration, partenaire, client) déployées et accessibles via un navigateur contrôlé par Playwright.
- **Émulateur Mobile** : émulateur Android ou simulateur iOS utilisé pour exécuter les tests d'intégration Flutter via Patrol et *integration_test*.
- **GitLab Pages** : service d'hébergement statique de GitLab utilisé pour publier le rapport Allure, accessible à toute l'équipe via une URL dédiée.

2.3 Conception de la pyramide de tests

La pyramide de tests est un modèle conceptuel fondamental en ingénierie logicielle, popularisé par Mike Cohn [?]. Elle recommande de structurer les tests en couches hiérarchiques, avec une base large de tests rapides et peu coûteux et un sommet étroit de tests lents mais plus réalistes.

Dans le cadre de notre projet, nous adoptons une pyramide à quatre couches, complétée par des tests de sécurité transversaux, comme l'illustre la figure 2.5.

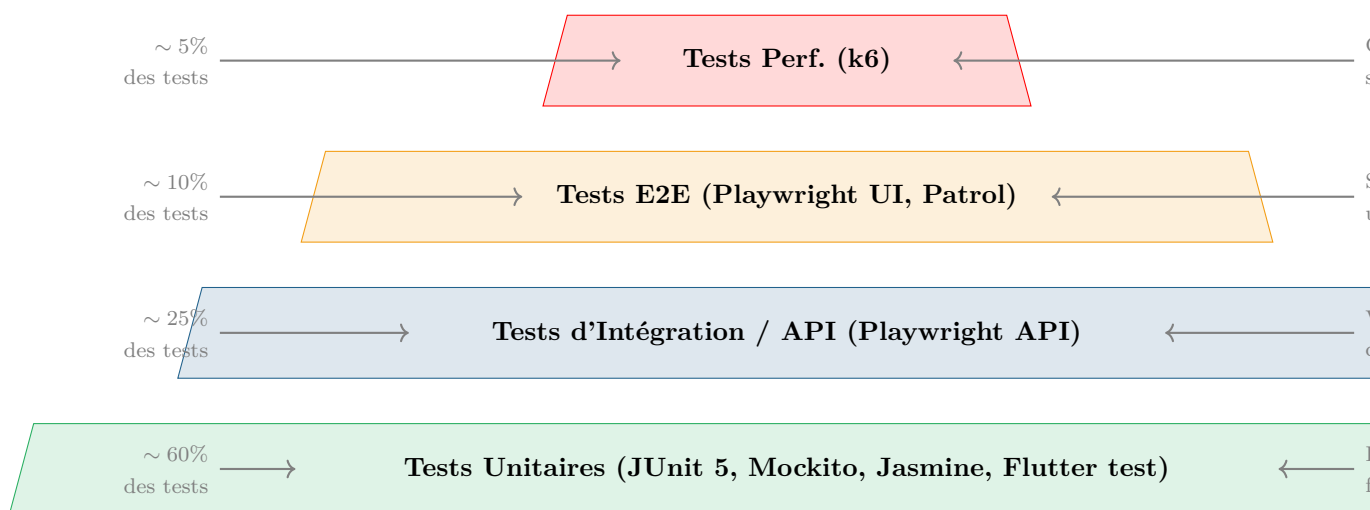


FIGURE 2.5 – Pyramide de tests adoptée pour le robot de tests automatisés

2.3.1 Couche 1 – Tests unitaires

Les tests unitaires constituent la base de la pyramide et représentent environ 60% de l'ensemble des tests. Ils vérifient le bon fonctionnement des unités de code individuelles (fonctions, méthodes, classes) de manière isolée.

- **Backend Spring Boot** : tests JUnit 5 avec Mockito pour moquer les dépendances. Couverture des services, des contrôleurs et des utilitaires.

- **Frontend Angular** : tests Jasmine/Karma pour les composants, les services et les pipes Angular.
- **Applications Flutter** : tests unitaires via le framework `flutter_test` pour les modèles, les *providers* et la logique métier.

Caractéristiques : exécution rapide (< 1 seconde par test), pas de dépendance externe, feedback immédiat au développeur.

2.3.2 Couche 2 – Tests d’intégration et API

Les tests d’intégration vérifient que les différents modules communiquent correctement entre eux. Dans notre contexte, ils se concentrent principalement sur les tests API REST.

- Vérification des endpoints REST (codes de retour, format de réponse, validation des données).
- Tests de contrat entre le frontend et le backend.
- Tests d’authentification et d’autorisation (vérification des rôles).
- Tests de la logique métier impliquant plusieurs services.

Outil : Playwright API Testing, qui permet d’envoyer des requêtes HTTP et de valider les réponses sans navigateur.

2.3.3 Couche 3 – Tests End-to-End (E2E)

Les tests E2E simulent le comportement réel de l’utilisateur en interagissant avec l’application à travers son interface graphique. Ils couvrent les flux métier complets.

- **Applications web** : Playwright contrôle un navigateur Chromium, Firefox ou WebKit pour simuler les interactions utilisateur (clics, saisie de texte, navigation).
- **Applications mobiles** : Patrol et `integration_test` permettent de tester les applications Flutter sur un émulateur ou un appareil physique, y compris les dialogues natifs (permissions, notifications).

Caractéristiques : tests lents mais réalistes, couvrant l’intégralité de la pile applicative.

2.3.4 Couche 4 – Tests de performance

Les tests de performance se situent au sommet de la pyramide. Ils évaluent le comportement de l’application sous charge et identifient les goulots d’étranglement.

- Tests de charge (*load testing*) : simulation de 100 à 500 utilisateurs simultanés.

- Tests de stress (*stress testing*) : montée progressive de la charge jusqu’à la rupture.
- Tests d’endurance (*soak testing*) : charge constante sur une durée prolongée.

Outil : k6, un outil de test de performance moderne, scriptable en JavaScript et optimisé pour l’intégration CI/CD.

2.3.5 Tests de sécurité – Couche transversale

Les tests de sécurité ne constituent pas une couche isolée mais traversent l’ensemble de la pyramide :

- **Niveau unitaire** : validation des entrées, échappement des caractères spéciaux.
- **Niveau API** : tests d’injection SQL, vérification des en-têtes de sécurité (CORS, CSP, HSTS).
- **Niveau E2E** : tests XSS, vérification du comportement face aux tentatives de contournement d’authentification.
- **Niveau performance** : résistance aux attaques DDoS basiques.

2.4 Plan de test global

Le plan de test global définit l’ensemble des scénarios de test à implémenter, classés par catégorie, type et priorité. Le tableau 2.3 présente les principaux scénarios identifiés.

TABLE 2.3 – Plan de test global du robot de tests automatisés

ID	Catégorie	Scénario	Type	Priorité	Outil
T01	Auth	Connexion admin valide	E2E	Haute	Playwright
T02	Auth	Connexion admin invalide	E2E	Haute	Playwright
T03	Auth	Inscription client	API	Haute	Playwright API
T04	Auth	Vérification OTP	API	Haute	Playwright API
T05	Auth	Expiration du token JWT	API	Moyenne	Playwright API
T06	Auth	Réinitialisation mot de passe	E2E	Moyenne	Playwright
T07	Commande	Création de commande complète	E2E	Haute	Playwright
T08	Commande	Acceptation par partenaire	E2E	Haute	Playwright

Suite page suivante

Table 2.3 – Suite de la page précédente

ID	Catégorie	Scénario	Type	Priorité	Outil
T09	Commande	Préparation de commande	API	Haute	Playwright API
T10	Commande	Affectation livreur	API	Haute	Playwright API
T11	Commande	Changement de statut livraison	API	Haute	Playwright API
T12	Commande	Annulation de commande	API	Moyenne	Playwright API
T13	Commande	Historique des commandes	E2E	Moyenne	Playwright
T14	Paiement	Paiement en ligne réussi	API	Haute	Playwright API
T15	Paiement	Paiement en ligne échoué	API	Haute	Playwright API
T16	Paiement	Paiement à la livraison	API	Haute	Playwright API
T17	Paiement	Calcul montant total	Unitaire	Haute	JUnit 5
T18	Partenaire	Ajout produit au menu	E2E	Haute	Playwright
T19	Partenaire	Modification prix produit	E2E	Moyenne	Playwright
T20	Partenaire	Désactivation produit	API	Moyenne	Playwright API
T21	Partenaire	Gestion catégories	E2E	Moyenne	Playwright
T22	Livreur	Tracking GPS temps réel	API	Haute	Playwright API
T23	Livreur	Calcul temps de livraison	Unitaire	Moyenne	JUnit 5
T24	Livreur	Affectation automatique	API	Haute	Playwright API
T25	Utilisateurs	Création compte admin	E2E	Haute	Playwright
T26	Utilisateurs	Modification rôle	API	Moyenne	Playwright API
T27	Utilisateurs	Désactivation compte	API	Moyenne	Playwright API
T28	Promotions	Création code promo	E2E	Moyenne	Playwright
T29	Promotions	Application code promo	API	Haute	Playwright API
T30	Promotions	Expiration code promo	Unitaire	Moyenne	JUnit 5
T31	Avis	Soumission avis client	API	Basse	Playwright API
T32	Avis	Affichage moyenne avis	E2E	Basse	Playwright
T33	Support	Création ticket support	API	Moyenne	Playwright API
T34	Support	Réponse ticket	E2E	Basse	Playwright
T35	Performance	Charge 100 utilisateurs	Perf	Haute	k6
T36	Performance	Charge 500 utilisateurs	Perf	Moyenne	k6
T37	Performance	Temps réponse API < 200ms	Perf	Haute	k6

Suite page suivante

Table 2.3 – Suite de la page précédente

ID	Catégorie	Scénario	Type	Priorité	Outil
T38	Sécurité	Injection SQL sur login	API	Haute	Playwright API
T39	Sécurité	XSS sur champs texte	E2E	Haute	Playwright
T40	Sécurité	Accès non autorisé API	API	Haute	Playwright API
T41	Mobile	Connexion app client	E2E	Haute	Patrol
T42	Mobile	Passage commande mobile	E2E	Haute	Patrol
T43	Mobile	Tracking livreur mobile	E2E	Haute	Patrol

Répartition des tests. La figure 2.4 montre la répartition des scénarios de test par type et par catégorie.

TABLE 2.4 – Répartition des scénarios par type de test

Type de test	Nombre de scénarios	Pourcentage
Tests unitaires	3	7%
Tests API	18	42%
Tests E2E	16	37%
Tests de performance	3	7%
Tests mobiles (E2E)	3	7%
Total	43	100%

Note : Les tests unitaires sont sous-représentés dans ce tableau car ils sont déjà présents dans les projets existants. Le robot de tests se concentre principalement sur les couches API, E2E et performance qui étaient absentes avant notre intervention.

2.5 Choix techniques

Le choix des outils de test est une décision stratégique qui impacte directement la productivité de l'équipe, la maintenabilité de la suite de tests et la qualité du feedback fourni. Nous justifions dans cette section chacun de nos choix en les confrontant aux alternatives existantes.

2.5.1 Playwright vs Cypress

Pour l'automatisation des tests web (interfaces Angular), nous avons comparé les deux frameworks leaders du marché : Playwright et Cypress. Le tableau 2.5 présente une

comparaison détaillée.

TABLE 2.5 – Comparaison Playwright vs Cypress

Critère	Playwright	Cypress
Multi-navigateur	Chromium, Firefox, WebKit	Chromium, Firefox (partiel), pas de WebKit
Tests API intégrés	Oui, natif (<code>request</code> context)	Oui, via <code>cy.request</code>
Parallélisation	Natif, gratuit	Payant (Cypress Dashboard)
Multi-onglets	Supporté nativement	Non supporté
Langages	TypeScript, JavaScript, Python, Java, C#	JavaScript, TypeScript uniquement
Vitesse d'exécution	Rapide (protocole CDP)	Moyenne (exécution dans le navigateur)
Intégration CI/CD	Excellente, images Docker officielles	Bonne, nécessite configuration
Auto-wait	Intelligent, configurable	Automatique, moins flexible
Communauté	Croissance rapide (Microsoft)	Large, mature
Licence	Apache 2.0 (entièrement gratuit)	Freemium (fonctions avancées payantes)

Justification du choix. Nous avons retenu **Playwright** pour les raisons suivantes :

1. **Support multi-navigateur complet** : la possibilité de tester sur Chromium, Firefox et WebKit assure une couverture maximale des navigateurs utilisés par les clients Speed Line.
2. **Tests API natifs** : Playwright permet de combiner tests UI et tests API dans un même framework, réduisant la complexité de la pile technologique.
3. **Parallélisation gratuite** : contrairement à Cypress qui nécessite un abonnement payant pour la parallélisation, Playwright offre cette fonctionnalité nativement.
4. **Performance supérieure** : Playwright communique avec le navigateur via le protocole Chrome DevTools Protocol (CDP), ce qui est plus rapide que l'approche de Cypress qui s'exécute à l'intérieur du navigateur.
5. **Intégration CI/CD excellente** : les images Docker officielles Playwright simplifient considérablement la configuration dans GitLab CI.

2.5.2 Patrol vs Appium

Pour les tests de l'application mobile Flutter, nous avons comparé Patrol (avec `integration_test`) et Appium. Le tableau 2.6 résume cette comparaison.

TABLE 2.6 – Comparaison Patrol vs Appium

Critère	Patrol	Appium
Spécifique Flutter	Oui, conçu pour Flutter	Non, générique (iOS, Android, Web)
Pont (<i>bridge</i>)	Aucun, exécution native	WebDriver bridge (latence)
Dialogues natifs	Gestion native (permissions, alertes)	Nécessite configuration complexe
Langage de test	Dart (même que l'application)	Java, Python, JavaScript, etc.
Vitesse	Très rapide (pas de sérialisation)	Lent (communication via pont)
Configuration	Minimale, intégré à Flutter	Complexe (serveur Appium, drivers)
Accès aux widgets	Direct, via <code>find.byType</code>	Via sélecteurs d'accessibilité
Communauté	En croissance	Très large, mature
Maintenance	Faible, lié au projet Flutter	Élevée, mises à jour fréquentes

Justification du choix. Nous avons retenu **Patrol** pour les raisons suivantes :

1. **Natif Flutter** : Patrol est conçu spécifiquement pour Flutter, ce qui élimine le surcoût du pont de communication (*bridge overhead*) d'Appium.
2. **Gestion des dialogues natifs** : Patrol gère nativement les dialogues système (permissions de géolocalisation, notifications), ce qui est critique pour l'application livreur qui utilise le GPS.
3. **Même langage** : les tests sont écrits en Dart, le même langage que l'application, ce qui facilite la collaboration entre développeurs et testeurs.
4. **Performance** : l'absence de sérialisation entre le test et l'application rend l'exécution significativement plus rapide.
5. **Simplicité de configuration** : pas besoin d'installer et de configurer un serveur Appium séparé.

2.5.3 k6 vs JMeter vs Gatling

Pour les tests de performance, nous avons évalué trois outils populaires : k6, Apache JMeter et Gatling. Le tableau 2.7 présente la comparaison.

TABLE 2.7 – Comparaison k6 vs JMeter vs Gatling

Critère	k6	JMeter	Gatling
Langage	JavaScript/TypeScript	XML (GUI)	Scala/Java
Interface	CLI (code-first)	GUI lourde	CLI + DSL
Empreinte mémoire	Faible (Go runtime)	Élevée (JVM)	Moyenne (JVM)
Intégration CI/CD	Excellente (CLI natif)	Complexe (GUI)	Bonne
Courbe d'apprentissage	Douce (JavaScript)	Moyenne	Raide (Scala)
Protocoles	HTTP, WebSocket, gRPC	HTTP, JDBC, FTP, JMS, etc.	HTTP, WebSocket
Rapports	Intégrés + export	Intégrés	HTML élégant
Conteneurisation	Image Docker légère	Image Docker lourde	Image Docker moyenne
Licence	AGPL v3 (gratuit)	Apache 2.0	Apache 2.0

Justification du choix. Nous avons retenu **k6** pour les raisons suivantes :

1. **Approche code-first en JavaScript** : les scripts de test sont écrits en JavaScript, un langage déjà maîtrisé par l'équipe, contrairement au Scala de Gatling ou à l'interface XML de JMeter.
2. **Légèreté** : k6 est compilé en Go et consomme très peu de mémoire, ce qui permet de l'exécuter dans un conteneur CI/CD sans surcoût.
3. **Intégration CI/CD native** : k6 est conçu dès le départ pour fonctionner en ligne de commande, ce qui facilite son intégration dans GitLab CI.
4. **Résultats exploitables** : k6 produit des métriques standardisées (temps de réponse, débit, taux d'erreur) exportables vers des outils de visualisation comme Grafana.

2.5.4 Allure vs HTML Reporter

Pour la génération de rapports de test, nous avons comparé Allure Report et le HTML Reporter natif de Playwright. Le tableau 2.8 présente cette comparaison.

TABLE 2.8 – Comparaison Allure Report vs HTML Reporter

Critère	Allure Report	HTML Reporter
Interface	Riche, interactive, navigation par catégorie	Simple, liste plate des tests
Historique	Oui, tendances sur plusieurs exécutions	Non, exécution courante uniquement
Captures d'écran	Intégrées avec navigation	Intégrées, moins ergonomiques
Traces réseau	Oui, avec détail des requêtes	Oui (trace viewer)
Catégorisation	Par fonctionnalité, sévérité, durée	Par fichier de test uniquement
Intégration multi-outils	JUnit, Playwright, k6, Flutter	Playwright uniquement
Personnalisation	Thèmes, plugins, environnements	Limitée
Hébergement	GitLab Pages, S3, serveur statique	Fichier local uniquement

Justification du choix. Nous avons retenu **Allure Report** pour les raisons suivantes :

1. **Rapport consolidé multi-outils** : Allure peut agréger les résultats de JUnit 5, Playwright, k6 et Flutter dans un rapport unique et cohérent.
2. **Historique et tendances** : la fonctionnalité d'historique permet de suivre l'évolution de la qualité au fil du temps, identifiant les régressions et les améliorations.
3. **Interface riche** : l'interface web interactive facilite l'analyse des résultats par les membres non techniques de l'équipe (chefs de projet, *product owners*).
4. **Intégration GitLab Pages** : le rapport peut être automatiquement publié sur GitLab Pages à chaque exécution du pipeline, rendant les résultats accessibles à toute l'équipe.

2.6 Architecture du robot de test

L'architecture du robot de test a été conçue pour être modulaire, maintenable et extensible. Elle suit une organisation en répertoires thématiques, chaque répertoire regroupant les tests et les ressources d'un même type.

2.6.1 Structure des répertoires

La structure globale du projet de tests est présentée ci-dessous :

```
1 tests/
2 |-- playwright/           # Tests web UI + API
3 |   |-- pages/            # Page Object Model
4 | |   |-- admin/          # Pages admin (Angular)
5 | |   |-- partner/        # Pages partenaire (Angular)
6 | |   |-- client/         # Pages client (Angular)
7 | |   |-- api/            # Tests API REST
8 | |   |-- e2e/            # Tests E2E navigateur
9 | |   |-- security/       # Tests de securite
10 | |   |-- fixtures/      # Donnees de test
11 | |   |-- utils/         # Utilitaires partages
12 | |   |-- playwright.config.ts
13 |-- mobile/              # Tests Flutter
14 |   |-- client_app/       # Tests app client
15 | |   |-- integration_test/
16 | |   |-- test/          # Tests unitaires
17 | |   |-- driver_app/    # Tests app livreur
18 | |   |-- integration_test/
19 | |   |-- test/          # Tests unitaires
20 |-- performance/         # Scripts k6
21 |   |-- scenarios/       # Scenarios de charge
22 |   |-- thresholds/      # Seuils de performance
23 |   |-- utils/           # Utilitaires k6
24 |-- seed/                # Script d'initialisation DB
25 |   |-- data/            # Donnees de seed
26 |   |-- scripts/         # Scripts SQL/API
27 |-- ci/                  # Configuration GitLab CI
28 |   |-- .gitlab-ci.yml   # Pipeline principal
29 |   |-- jobs/            # Jobs specifiques
30 |-- docs/                # Documentation
31 |   |-- test-plan.md     # Plan de test detaille
32 |   |-- manual/          # Scenarios manuels
```

Listing 2.1 – Structure des répertoires du robot de tests

2.6.2 Architecture logique

La figure 2.6 présente l'architecture logique du robot de tests, montrant les interactions entre les différents modules.

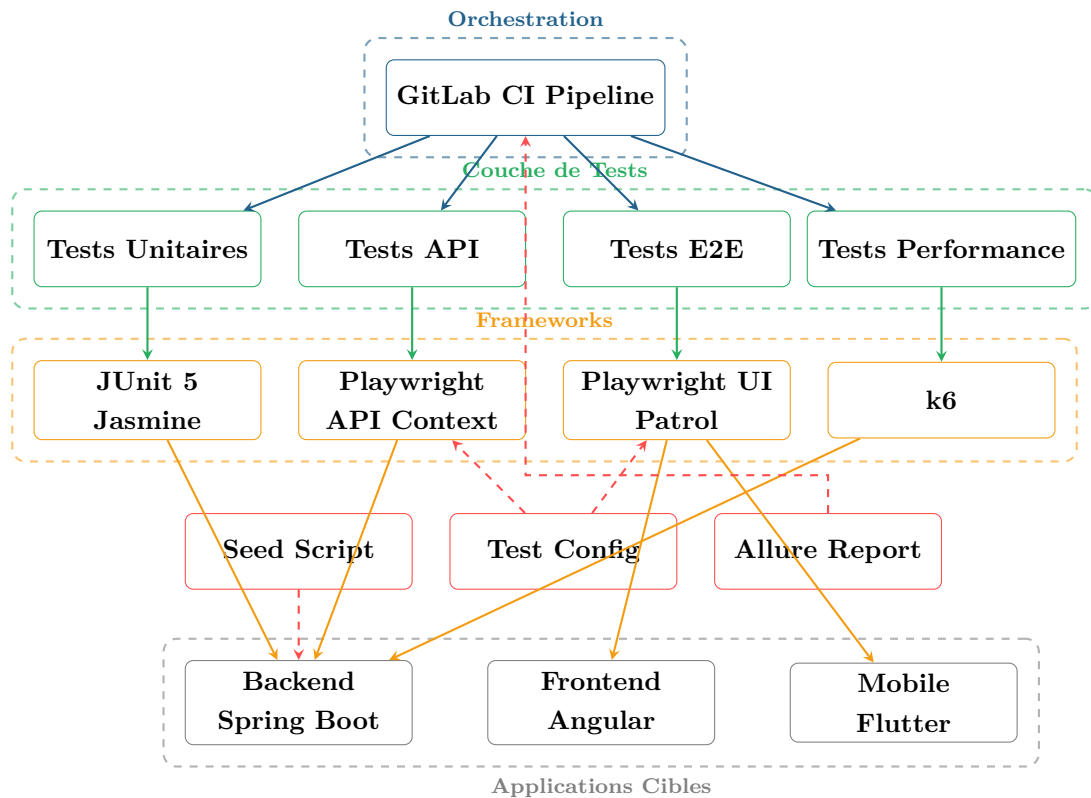


FIGURE 2.6 – Architecture logique du robot de tests

2.6.3 Description des modules

Module Playwright (tests web et API). Ce module constitue le cœur du robot de tests. Il est organisé selon le patron *Page Object Model*, avec une séparation claire entre :

- Les **pages** (`pages/`) : classes décrivant les éléments et les actions de chaque page web.
- Les **tests API** (`api/`) : scénarios vérifiant les endpoints REST de l'API Spring Boot.
- Les **tests E2E** (`e2e/`) : scénarios simulant le parcours utilisateur complet sur les interfaces Angular.
- Les **tests de sécurité** (`security/`) : vérifications des vulnérabilités courantes (injection SQL, XSS).

Module Mobile (tests Flutter). Ce module contient les tests pour les deux applications mobiles Flutter :

- L'application **client** : tests du parcours de commande, du tracking et du paiement.
- L'application **livreur** : tests de la réception des commandes, du tracking GPS et de la confirmation de livraison.

Les tests utilisent le framework `integration_test` de Flutter, complété par Patrol pour la gestion des dialogues natifs (permissions de géolocalisation, notifications push).

Module Performance (scripts k6). Ce module contient les scénarios de test de charge écrits en JavaScript pour k6 :

- Simulation de **charge normale** : 100 utilisateurs simultanés pendant 5 minutes.
- Simulation de **pic de charge** : montée progressive de 0 à 500 utilisateurs.
- Test d'**endurance** : 50 utilisateurs constants pendant 30 minutes.

Module Seed (initialisation de la base de données). Le script de *seed* garantit un état initial reproductible de la base de données avant chaque exécution de tests. Il crée :

- Des comptes de test pour chaque rôle (admin, client, partenaire, livreur).
- Des partenaires avec leurs menus et catégories de produits.
- Des commandes dans différents états pour tester les transitions.
- Des codes promotionnels actifs et expirés.

Module CI (configuration GitLab CI). La configuration du pipeline GitLab CI orchestre l'exécution séquentielle des étapes :

1. **Stage seed** : initialisation de la base de données.
2. **Stage unit** : exécution des tests unitaires (JUnit 5, Jasmine, Flutter test).
3. **Stage api** : exécution des tests API Playwright.
4. **Stage e2e** : exécution des tests E2E Playwright et Patrol.
5. **Stage perf** : exécution des tests de performance k6.
6. **Stage report** : génération et publication du rapport Allure.

Conclusion

Dans ce chapitre, nous avons mené une analyse approfondie des besoins fonctionnels et non fonctionnels du robot de tests automatisés. La modélisation UML nous a permis de

visualiser les interactions entre les acteurs, le flux d'exécution des tests, l'architecture logicielle et l'infrastructure de déploiement.

Nous avons justifié nos choix techniques (Playwright, Patrol, k6, Allure) en les confrontant aux alternatives du marché, en nous appuyant sur des critères objectifs de performance, de maintenabilité et d'intégration CI/CD.

La conception de la pyramide de tests à quatre couches, complétée par des tests de sécurité transversaux, assure une couverture optimale tout en maîtrisant le temps d'exécution. Le plan de test global, avec ses 43 scénarios répartis sur toutes les catégories métier, constitue la feuille de route pour la phase de réalisation que nous abordons dans le chapitre suivant.

Chapitre 3

Réalisation

Introduction

Ce chapitre constitue le volet technique principal de notre travail. Il présente en détail la mise en œuvre concrète du robot de tests automatisés pour la plateforme Speed-Line. Nous y décrivons l’environnement de développement retenu, puis nous détaillons l’implémentation de chaque couche de la pyramide de tests : tests unitaires, tests API, tests de bout en bout (E2E), et tests de performance. Nous abordons ensuite les aspects transversaux que sont les tests de sécurité, le *seed* de la base de données, l’intégration CI/CD et le reporting avec Allure.

3.1 Environnement de développement

La mise en place du robot de tests nécessite un ensemble d’outils couvrant les différents niveaux de la pyramide de tests. Le tableau 3.1 récapitule les principaux outils utilisés, leur version ainsi que leur rôle au sein du projet.

TABLE 3.1 – Environnement de développement et outils utilisés

Outil	Version	Rôle
Node.js	24.x	Environnement d'exécution JavaScript pour les tests Playwright et k6
npm	11.x	Gestionnaire de paquets Node.js
Java (OpenJDK)	17	Langage du backend Spring Boot (microservices)
Maven	3.9	Outil de build et de gestion des dépendances Java
Playwright	1.49	Framework de tests API et E2E pour les applications web
JUnit 5	5.x	Framework de tests unitaires Java
Mockito	5.x	Bibliothèque de mocking pour les tests unitaires Java
Jasmine + Karma	—	Frameworks de tests unitaires pour les frontends Angular
Patrol	3.13	Framework de tests d'intégration pour les applications Flutter
k6	1.7	Outil de tests de charge et de performance
Allure	2.27	Générateur de rapports de tests interactifs
Docker	—	Conteneurisation des services pour l'environnement de test
Git + GitLab CI	—	Gestion de versions et intégration continue
VS Code	—	Éditeur de code pour TypeScript, JavaScript et Dart
IntelliJ IDEA	—	IDE pour le développement Java (backend)
Flutter	3.41	Framework de développement des applications mobiles
Angular	17	Framework de développement des interfaces web (admin-panel, partner-dashboard)

Le choix de ces outils s'inscrit dans une logique de cohérence avec la pile technologique existante du projet SpeedLine. Les tests unitaires backend reposent sur JUnit 5 et Mockito, standards de l'écosystème Spring Boot. Pour les tests API et E2E web, Playwright a été retenu pour sa rapidité d'exécution, sa fiabilité et son support multi-navigateurs. Les applications mobiles Flutter sont testées avec Patrol, qui offre la possibilité d'interagir avec les dialogues natifs du système d'exploitation. Enfin, k6 a été choisi pour les tests de performance grâce à sa syntaxe JavaScript familière et ses métriques intégrées.

3.2 Mise en place des tests unitaires (Couche 1)

Les tests unitaires constituent la base de notre pyramide de tests. Ils visent à vérifier le comportement isolé de chaque composant logiciel, en simulant les dépendances externes à l'aide de *mocks*. Cette approche garantit des tests rapides, déterministes et indépendants de l'infrastructure.

3.2.1 Tests unitaires backend (JUnit 5 + Mockito)

Approche adoptée

Pour chaque service du backend, nous appliquons la même méthodologie :

1. **Isolation des dépendances** : les repositories, clients Feign et services externes sont simulés (*mockés*) avec Mockito via l'annotation `@Mock`.
2. **Injection automatique** : le service sous test est instancié avec `@InjectMocks`, ce qui injecte automatiquement les mocks.
3. **Organisation en classes imbriquées** : les tests sont groupés par méthode testée à l'aide de l'annotation `@Nested`, améliorant la lisibilité des rapports.
4. **Assertions expressives** : nous utilisons AssertJ pour des assertions fluides et lisibles.

Exemple : AuthServiceImplTest

Le listing 3.1 illustre la structure d'un test unitaire pour le service d'authentification. Ce test vérifie le scénario d'inscription réussie d'un nouveau client.

```
1 @ExtendWith(MockitoExtension.class)
2 class AuthServiceImplTest {
3
4     @Mock private UserRepository userRepo;
5     @Mock private PasswordEncoder passwordEncoder;
6     @Mock private JwtTokenProvider tokenProvider;
7     @Mock private AuthenticationManager authenticationManager;
8     @Mock private UserServiceClient userServiceClient;
9
10    @InjectMocks
11    private AuthServiceImpl authService;
12
13    @Nested
14    @DisplayName("register()")
15    class RegisterTests {
16
17        @Test
```

```
18         @DisplayName("Should register a new customer successfully")
19         void register_success_customer() {
20             RegisterRequest request = new RegisterRequest();
21             request.setEmail("new@example.com");
22             request.setPassword("Password123");
23             request.setFirstName("Jane");
24             request.setLastName("Doe");
25             request.setPhoneNumber("+21655443322");
26             request.setRole(Role.CUSTOMER);
27
28             when(userRepo.existsByEmailIgnoreCase("new@example.
29 com"))
30                 .thenReturn(false);
31             when(passwordEncoder.encode("Password123"))
32                 .thenReturn("encoded-pass");
33             when(userRepo.save(any(User.class)))
34                 .thenAnswer(invocation -> {
35                     User u = invocation.getArgument(0);
36                     u.setId(10L);
37                     return u;
38                 });
39
40             authService.register(request);
41
42             ArgumentCaptor<User> captor =
43                 ArgumentCaptor.forClass(User.class);
44             verify(userRepo).save(captor.capture());
45             User saved = captor.getValue();
46
47             assertThat(saved.getEmail())
48                 .isEqualTo("new@example.com");
49             assertThat(saved.getPassword())
50                 .isEqualTo("encoded-pass");
51             assertThat(saved.getRole())
52                 .isEqualTo(Role.CUSTOMER);
53             assertThat(saved.getStatus())
54                 .isEqualTo(UserStatus.PENDING);
55             verify(userServiceClient).createCustomer(any());
56         }
57     }
```

Listing 3.1 – Test d’inscription – AuthServiceImplTest.java

On note l’utilisation de `ArgumentCaptor` pour capturer l’objet `User` passé à la méthode `save()`, permettant de vérifier finement les valeurs assignées lors de l’inscription. La

méthode `verify()` s'assure que l'appel au service distant `userServiceClient` a bien été effectué.

Exemple : JwtServiceTest

Le listing 3.2 présente les tests du fournisseur de jetons JWT. Contrairement aux tests précédents, ces tests n'utilisent pas de mocks car le `JwtTokenProvider` est testé avec sa logique réelle de génération et de validation de jetons.

```
1 class JwtServiceTest {
2
3     private JwtTokenProvider jwtTokenProvider;
4     private static final String TEST_SECRET =
5         "my-super-secret-key-for-testing-jwt-tokens-"
6         + "must-be-long-enough-256-bits";
7     private static final long EXPIRATION_MS = 3600000L;
8
9     private User testUser;
10
11     @BeforeEach
12     void setUp() {
13         jwtTokenProvider = new JwtTokenProvider();
14         ReflectionTestUtils.setField(
15             jwtTokenProvider, "jwtSecret", TEST_SECRET);
16         ReflectionTestUtils.setField(
17             jwtTokenProvider, "jwtExpirationMs", EXPIRATION_MS);
18
19         testUser = User.builder()
20             .id(42L)
21             .email("test@example.com")
22             .role(Role.CUSTOMER)
23             .status(UserStatus.ACTIVE)
24             .build();
25     }
26
27     @Nested
28     @DisplayName("generateToken()")
29     class GenerateTokenTests {
30
31         @Test
32         @DisplayName("Should embed email as subject in token")
33         void generateToken_containsEmailAsSubject() {
34             String token = jwtTokenProvider.generateToken(
35                 testUser);
36             String email = jwtTokenProvider
37                 .getEmailFromToken(token);
38             assertThat(email).isEqualTo("test@example.com");
39         }
40     }
41 }
```

```

38         }
39
40         @Test
41         @DisplayName("Should embed role claim in token")
42         void generateToken_containsRole() {
43             String token = jwtTokenProvider.generateToken(
44                 testUser);
45             String role = jwtTokenProvider
46                 .getRoleFromToken(token);
47             assertThat(role).isEqualTo("CUSTOMER");
48         }
49     }

```

Listing 3.2 – Test de génération JWT – JwtServiceTest.java

Bilan des tests unitaires backend

Le tableau 3.2 présente la répartition des tests unitaires par microservice. Au total, **419 tests** ont été implémentés, couvrant l'ensemble des services métier de la plateforme.

TABLE 3.2 – Répartition des tests unitaires backend par microservice

Microservice	Nombre de tests	Classes testées
auth-service	100	AuthServiceImpl, JwtService, AuthController
order-service	87	OrderService, CartService, PriceCalculation
user-service	57	CustomerService, CourierService
delivery-service	41	DispatchCycleService, EligibilityFilter, Solvers
payment-service	30	PaymentServiceImpl, WalletService, RefundService
location-service	29	ZoneServiceImpl, GeolocationServiceImpl
promotion-service	20	PromotionApplicationService, Validators
notification-service	18	NotificationServiceImpl
analytics-service	16	AnalyticsServiceImpl, ReportServiceImpl
support-service	11	SupportTicketServiceImpl
review-service	10	ReviewServiceImpl
Total	419	

Les services les plus critiques — **auth-service** et **order-service** — concentrent le plus grand nombre de tests (100 et 87 respectivement), reflétant leur complexité fonctionnelle et leur importance dans le flux métier de la plateforme.

3.2.2 Tests unitaires frontend (Jasmine + Karma)

Les applications web Angular (`admin-panel` et `partner-dashboard`) disposent de tests unitaires exécutés par Karma avec le framework d'assertions Jasmine. Ces tests vérifient le comportement des services, des gardes de route (*guards*) et des intercepteurs HTTP.

Exemple : AuthService (admin-panel)

Le listing 3.3 montre un extrait des tests du service d'authentification du panneau d'administration. Le service `ApiService` est simulé avec `jasmine.createSpyObj` pour isoler la logique d'authentification.

```
1 describe('AuthService', () => {
2   let service: AuthService;
3   let apiServiceSpy: jasmine.SpyObj<ApiService>;
4
5   const mockLoginResponse = {
6     access_token: 'eyJhbGciOiJIUzI1NiJ9...',
7     user: {
8       id: '1',
9       email: 'admin@speedline-test.com',
10      role: 'SUPER_ADMIN',
11    },
12  };
13
14  beforeEach(() => {
15    apiServiceSpy = jasmine.createSpyObj('ApiService',
16      ['get', 'post', 'put']);
17    TestBed.configureTestingModule({
18      imports: [HttpClientTestingModule],
19      providers: [
20        AuthService,
21        { provide: ApiService, useValue: apiServiceSpy },
22      ],
23    });
24    service = TestBed.inject(AuthService);
25    localStorage.clear();
26    sessionStorage.clear();
27  });
28
29  describe('login', () => {
30    it('should store tokens on successful login', (done) => {
31      apiServiceSpy.post.and.returnValue(
32        of(mockLoginResponse));
33      service.login({
34        email: 'admin@test.com',
35        password: 'pass',
```

```
36     rememberMe: false
37   }).subscribe({
38     next: () => {
39       expect(service.isLoggedIn()).toBeTrue();
40       expect(service.currentUser()?.email)
41         .toBe('admin@speedline-test.com');
42       done();
43     },
44   });
45 });
46
47 it('should reject non-admin roles', (done) => {
48   const customerResponse = {
49     ...mockLoginResponse,
50     user: { ...mockLoginResponse.user,
51       role: 'CUSTOMER' },
52   };
53   apiServiceSpy.post.and.returnValue(
54     of(customerResponse));
55   service.login({
56     email: 'customer@test.com',
57     password: 'pass',
58     rememberMe: false
59   }).subscribe({
60     error: (err) => {
61       expect(err.message).toBe('ACCESS_DENIED');
62       done();
63     },
64   });
65 });
66 });
67 });
```

Listing 3.3 – Test du service d’authentification Angular – auth.service.spec.ts

Exemple : AuthGuard

Le listing 3.4 présente les tests du garde de route `AuthGuard`, qui protège les pages nécessitant une authentification.

```
1 describe('AuthGuard', () => {
2   let authServiceSpy: jasmine.SpyObj<AuthService>;
3   let routerSpy: jasmine.SpyObj<Router>;
4
5   beforeEach(() => {
6     authServiceSpy = jasmine.createSpyObj('AuthService',
7       ['isAuthenticated']);
```

```
8   routerSpy = jasmine.createSpyObj('Router', ['navigate']);
9   TestBed.configureTestingModule({
10     providers: [
11       { provide: AuthService, useValue: authServiceSpy },
12       { provide: Router, useValue: routerSpy },
13     ],
14   });
15 });
16
17 it('should allow access when authenticated', () => {
18   authServiceSpy.isAuthenticated.and.returnValue(true);
19   const result = TestBed.runInInjectionContext(
20     () => authGuard(mockRoute, mockState));
21   expect(result).toBeTrue();
22 });
23
24 it('should redirect to login when not authenticated',
25   () => {
26     authServiceSpy.isAuthenticated.and.returnValue(false);
27     const result = TestBed.runInInjectionContext(
28       () => authGuard(mockRoute, mockState));
29     expect(result).toBeFalse();
30     expect(routerSpy.navigate).toHaveBeenCalledWith(
31       ['/auth/login'],
32       { queryParams: { returnUrl: '/dashboard' } });
33   });
34 });
```

Listing 3.4 – Test du garde d'authentification – auth.guard.spec.ts

Exemple : AuthInterceptor

Le listing 3.5 montre les tests de l'intercepteur HTTP, qui ajoute automatiquement le jeton JWT aux requêtes sortantes.

```
1 describe('authInterceptor', () => {
2   let httpClient: HttpClient;
3   let httpMock: HttpTestingController;
4   let authServiceSpy: jasmine.SpyObj<AuthService>;
5
6   beforeEach(() => {
7     authServiceSpy = jasmine.createSpyObj('AuthService',
8       ['getToken']);
9     TestBed.configureTestingModule({
10       providers: [
11         provideHttpClient(
12           withInterceptors([authInterceptor])),
```



```

13     provideHttpClientTesting(),
14     { provide: AuthService,
15       useValue: authServiceSpy },
16   ],
17 });
18 httpClient = TestBed.inject(HttpClient);
19 httpMock = TestBed.inject(HttpTestingController);
20 });
21
22 it('should add Authorization header', () => {
23     authServiceSpy.getToken.and.returnValue('my-jwt');
24     httpClient.get('/api/v1/orders').subscribe();
25     const req = httpMock.expectOne('/api/v1/orders');
26     expect(req.request.headers.get('Authorization'))
27       .toBe('Bearer my-jwt');
28 });
29
30 it('should skip auth endpoints', () => {
31     authServiceSpy.getToken.and.returnValue('my-jwt');
32     httpClient.get('/api/v1/auth/login').subscribe();
33     const req = httpMock.expectOne('/api/v1/auth/login');
34     expect(req.request.headers.has('Authorization'))
35       .toBeFalse();
36 });
37 });

```

Listing 3.5 – Test de l’intercepteur HTTP – auth.interceptor.spec.ts

Bilan des tests unitaires frontend

Le tableau 3.3 présente le résultat des tests unitaires frontend ainsi que la couverture de code mesurée par Istanbul via Karma.

TABLE 3.3 – Résultats des tests unitaires frontend

Application	Nombre de tests	Couverture de code
admin-panel	26	57,78%
partner-dashboard	8	34,82%
Total	34	—

La couverture de 57,78% pour l’admin-panel couvre les services critiques (authentification, RBAC, intercepteurs). Le partner-dashboard, avec 34,82%, se concentre sur les flux d’authentification essentiels. Ces taux constituent une base solide, avec un potentiel d’amélioration lors des itérations futures.

3.3 Mise en place des tests API (Couche 2)

Les tests API constituent la deuxième couche de notre pyramide. Ils vérifient le comportement des microservices à travers leurs points d'entrée REST, en envoyant des requêtes HTTP réelles à l'*API Gateway* et en validant les réponses.

3.3.1 Architecture des tests API

Pour faciliter l'écriture et la maintenance des tests API, nous avons développé une classe utilitaire `ApiClient` qui encapsule la logique d'authentification et de requêtage. Le listing A.2 présente cette classe.

```
1 import { APIRequestContext, request } from '@playwright/test';
2
3 export class ApiClient {
4   private baseUrl: string;
5   private token: string | null = null;
6
7   constructor(baseUrl?: string) {
8     this.baseUrl = baseUrl
9       || process.env.API_GATEWAY_URL
10      || 'http://localhost:8080';
11   }
12
13   async login(email: string, password: string)
14     : Promise<string> {
15     const ctx = await request.newContext(
16       { baseUrl: this.baseUrl });
17     const response = await ctx.post('/api/auth/login', {
18       data: { email, password },
19     });
20     if (!response.ok()) {
21       throw new Error(
22         'Login failed: ${response.status()}');
23     }
24     const body = await response.json();
25     this.token = body.accessToken || body.token;
26     await ctx.dispose();
27     return this.token!;
28   }
29
30   async authenticatedContext()
31     : Promise<APIRequestContext> {
32     if (!this.token) {
33       throw new Error('Not logged in.');
```

```
35     return request.newContext({
36         baseUrl: this.baseUrl,
37         extraHTTPHeaders: {
38             Authorization: `Bearer ${this.token}`,
39         },
40     });
41 }
42
43 async get(path: string) {
44     const ctx = await this.authenticatedContext();
45     const res = await ctx.get(path);
46     await ctx.dispose();
47     return res;
48 }
49
50 async post(path: string, data: unknown) {
51     const ctx = await this.authenticatedContext();
52     const res = await ctx.post(path, { data });
53     await ctx.dispose();
54     return res;
55 }
56 }
```

Listing 3.6 – Classe utilitaire ApiClient – api-client.ts

Cette classe offre plusieurs avantages :

- **Centralisation de l’authentification** : un seul appel à `login()` suffit pour obtenir un jeton réutilisable.
- **Gestion automatique des en-têtes** : le jeton JWT est automatiquement ajouté à chaque requête via `authenticatedContext()`.
- **Nettoyage des ressources** : chaque contexte Playwright est libéré après utilisation avec `dispose()`.

3.3.2 Tests par service

Tests du service d’authentification

Le listing 3.7 présente les tests API du service d’authentification. Ces tests vérifient les scénarios de connexion, de validation de jeton et de gestion des erreurs.

```
1 test.describe('API - Auth Service', () => {
2     test('POST /api/auth/login - valid credentials',
3         async ({ request }) => {
4             const response = await request.post(
5                 `${API_BASE}/api/auth/login`, {
```

```
6     data: {
7       email: TEST_ACCOUNTS.admin.email,
8       password: TEST_ACCOUNTS.admin.password,
9     },
10  });
11  expect(response.status()).toBe(200);
12  const body = await response.json();
13  expect(body.accessToken || body.token).toBeTruthy();
14  });
15
16  test('POST /api/auth/login - invalid credentials',
17    async ({ request }) => {
18    const response = await request.post(
19      `${API_BASE}/api/auth/login`, {
20      data: {
21        email: 'nonexistent@test.com',
22        password: 'WrongPassword123!',
23      },
24    });
25    expect(response.status())
26      .toBeGreaterThanOrEqual(400);
27    expect(response.status()).toBeLessThan(500);
28  });
29
30  test('GET /api/auth/me - user info with valid token',
31    async ({ request }) => {
32    const loginRes = await request.post(
33      `${API_BASE}/api/auth/login`, {
34      data: {
35        email: TEST_ACCOUNTS.admin.email,
36        password: TEST_ACCOUNTS.admin.password,
37      },
38    });
39    const { accessToken } = await loginRes.json();
40    const meRes = await request.get(
41      `${API_BASE}/api/auth/me`, {
42      headers: {
43        Authorization: `Bearer ${accessToken}`,
44      },
45    });
46    expect(meRes.status()).toBe(200);
47  });
```

Listing 3.7 – Tests API du service d'authentification – auth.api.spec.ts

Tests du service de commandes

Le listing 3.8 illustre les tests CRUD du service de commandes, incluant la pagination et la création de commandes.

```
1 test.describe('API - Order Service', () => {
2   let api: ApiClient;
3   let customerApi: ApiClient;
4
5   test.beforeAll(async () => {
6     api = new ApiClient();
7     await api.login(TEST_ACCOUNTS.admin.email,
8       TEST_ACCOUNTS.admin.password);
9     customerApi = new ApiClient();
10    await customerApi.login(TEST_ACCOUNTS.customer.email,
11      TEST_ACCOUNTS.customer.password);
12  });
13
14  test('GET /api/orders - admin should list orders',
15    async () => {
16    const response = await api.get('/api/orders');
17    expect(response.status()).toBe(200);
18    const body = await response.json();
19    expect(Array.isArray(body.content || body))
20      .toBeTruthy();
21  });
22
23  test('POST /api/orders - customer should create order',
24    async () => {
25    const response = await customerApi.post(
26      '/api/orders', {
27      partnerId: 'test-partner-id',
28      items: TEST_ORDER.items,
29      deliveryAddress: TEST_ORDER.deliveryAddress,
30      paymentMethod: TEST_ORDER.paymentMethod,
31    });
32    expect([200, 201, 400])
33      .toContain(response.status());
34  });
35 });
```

Listing 3.8 – Tests API du service de commandes – orders.api.spec.ts

Test du cycle de vie complet d'une commande

Le listing 3.9 présente le test E2E API le plus complet : le cycle de vie intégral d'une commande, de la création à la livraison. Ce test implique quatre acteurs distincts

(client, partenaire, coursier, administrateur) et valide les transitions d'état.

```
1 test.describe('API - E2E Order Lifecycle', () => {
2   let adminApi: ApiClient;
3   let customerApi: ApiClient;
4   let partnerApi: ApiClient;
5   let courierApi: ApiClient;
6
7   test.beforeAll(async () => {
8     adminApi = new ApiClient();
9     await adminApi.login(TEST_ACCOUNTS.admin.email,
10       TEST_ACCOUNTS.admin.password);
11     customerApi = new ApiClient();
12     await customerApi.login(TEST_ACCOUNTS.customer.email,
13       TEST_ACCOUNTS.customer.password);
14     partnerApi = new ApiClient();
15     await partnerApi.login(TEST_ACCOUNTS.partner.email,
16       TEST_ACCOUNTS.partner.password);
17     courierApi = new ApiClient();
18     await courierApi.login(TEST_ACCOUNTS.courier.email,
19       TEST_ACCOUNTS.courier.password);
20   });
21
22   test('full order lifecycle: create -> accept '
23     + '-> pickup -> deliver', async () => {
24     // 1. Get available partners
25     const partnersRes = await customerApi.get(
26       '/api/partners?status=ACTIVE');
27     const partners = await partnersRes.json();
28     const partnerId = partners.content[0].id;
29
30     // 2. Create order
31     const createRes = await customerApi.post(
32       '/api/orders', {
33       partnerId,
34       items: TEST_ORDER.items,
35       deliveryAddress: TEST_ORDER.deliveryAddress,
36       paymentMethod: TEST_ORDER.paymentMethod,
37     });
38     const order = await createRes.json();
39
40     // 3. Partner accepts order
41     // 4. Courier picks up
42     // 5. Courier delivers
43     // 6. Order completed
44   });
45 });
```

Listing 3.9 – Test du cycle de vie complet – e2e-order-flow.api.spec.ts

Bilan des tests API

Le tableau 3.4 récapitule l'ensemble des fichiers de tests API et le nombre de cas de test par catégorie. Au total, **104 tests API** couvrent les 17 fichiers de spécifications.

TABLE 3.4 – Répartition des tests API par fichier de spécification

Fichier de test	Nombre de tests
auth.api.spec.ts	8
orders.api.spec.ts	6
users.api.spec.ts	6
support.api.spec.ts	6
partners.api.spec.ts	5
delivery.api.spec.ts	5
notifications.api.spec.ts	5
reviews.api.spec.ts	5
analytics.api.spec.ts	5
payments.api.spec.ts	4
location.api.spec.ts	4
promotions.api.spec.ts	3
e2e-order-flow.api.spec.ts	2
contracts.api.spec.ts	9
smoke.api.spec.ts	8
security.api.spec.ts	16
websocket.api.spec.ts	8
Total (17 fichiers)	104

Les tests de sécurité représentent la catégorie la plus fournie avec 16 tests, reflétant l'importance accordée à la sécurisation des API. Les tests **contracts** vérifient la conformité des contrats d'API (formats de réponse, codes HTTP), tandis que les tests **smoke** servent de vérification rapide de la disponibilité des services.

3.4 Mise en place des tests E2E (Couche 3)

Les tests de bout en bout (*End-to-End*) constituent la troisième couche de la pyramide. Ils simulent le comportement réel des utilisateurs en interagissant avec les interfaces

graphiques web et mobiles.

3.4.1 Tests Web (Playwright)

Le patron Page Object Model

Pour structurer les tests E2E web, nous adoptons le patron *Page Object Model* (POM). Chaque page de l'application est représentée par une classe TypeScript qui encapsule les localisateurs d'éléments et les actions utilisateur. Le listing 3.10 illustre cette approche pour la page de connexion de l'administration.

```
1 import { Page, Locator, expect } from '@playwright/test';
2
3 export class AdminLoginPage {
4   readonly page: Page;
5   readonly emailInput: Locator;
6   readonly passwordInput: Locator;
7   readonly loginButton: Locator;
8   readonly errorMessage: Locator;
9
10  constructor(page: Page) {
11    this.page = page;
12    this.emailInput = page.getByLabel(/email/i);
13    this.passwordInput = page.getByLabel(/password/i);
14    this.loginButton = page.getByRole('button',
15      { name: /sign in|login|connexion/i });
16    this.errorMessage = page.locator(
17      '[class*="error"]', [class*="alert-danger"]);
18  }
19
20  async goto() {
21    await this.page.goto('/auth/login');
22  }
23
24  async login(email: string, password: string) {
25    await this.emailInput.fill(email);
26    await this.passwordInput.fill(password);
27    await this.loginButton.click();
28  }
29
30  async expectDashboardRedirect() {
31    await this.page.waitForURL('**/dashboard',
32      { timeout: 15_000 });
33    await expect(this.page).toHaveURL(/dashboard/);
34  }
35
36  async expectError() {
```



```
37     await expect(this.errorMessage.first())
38       .toBeVisible({ timeout: 5_000 });
39   }
40 }
```

Listing 3.10 – Page Object – AdminLoginPage

Les avantages de cette approche sont multiples :

- **Maintenabilité** : si un sélecteur change, une seule classe doit être modifiée.
- **Réutilisabilité** : les actions (connexion, navigation) sont réutilisées dans plusieurs tests.
- **Lisibilité** : les tests décrivent des intentions utilisateur plutôt que des interactions DOM de bas niveau.

Tests d'authentification (admin-panel)

Le listing 3.11 présente les tests E2E d'authentification de l'administration, utilisant le `AdminLoginPage` défini précédemment.

```
1 test.describe('Admin Panel - Authentication', () => {
2   test.use({ storageState:
3     { cookies: [], origins: [] } });
4
5   test('should display login page', async ({ page }) => {
6     const loginPage = new AdminLoginPage(page);
7     await loginPage.goto();
8     await expect(loginPage.emailInput).toBeVisible();
9     await expect(loginPage.passwordInput).toBeVisible();
10    await expect(loginPage.loginButton).toBeVisible();
11  });
12
13  test('should login with valid admin credentials',
14    async ({ page }) => {
15    const loginPage = new AdminLoginPage(page);
16    await loginPage.goto();
17    await loginPage.login(
18      TEST_ACCOUNTS.admin.email,
19      TEST_ACCOUNTS.admin.password);
20    await loginPage.expectDashboardRedirect();
21  });
22
23  test('should show error with invalid credentials',
24    async ({ page }) => {
25    const loginPage = new AdminLoginPage(page);
26    await loginPage.goto();
27    await loginPage.login(
```

```
28     'invalid@test.com', 'WrongPassword123!');
29     await loginPage.expectError();
30   });
31
32   test('should redirect unauthenticated users',
33     async ({ page }) => {
34     await page.goto('/dashboard');
35     await expect(page).toHaveURL(/auth/);
36   });
37 });
```

Listing 3.11 – Tests E2E d'authentification – auth.spec.ts

Tests du tableau de bord

Le listing 3.12 illustre les tests de navigation du tableau de bord d'administration, vérifiant le chargement des pages et la navigation latérale.

```
1 test.describe('Admin Panel - Dashboard', () => {
2   test('should load dashboard after login',
3     async ({ page }) => {
4     const dashboard = new AdminDashboardPage(page);
5     await dashboard.goto();
6     await dashboard.expectLoaded();
7   });
8
9   test('should display sidebar navigation',
10    async ({ page }) => {
11    const dashboard = new AdminDashboardPage(page);
12    await dashboard.goto();
13    await expect(dashboard.sidebar).toBeVisible();
14  });
15
16  test('should navigate to orders',
17    async ({ page }) => {
18    const dashboard = new AdminDashboardPage(page);
19    await dashboard.goto();
20    await dashboard.navigateTo('orders');
21    await expect(page).toHaveURL(/orders/);
22  });
23
24  test('should navigate to partners',
25    async ({ page }) => {
26    const dashboard = new AdminDashboardPage(page);
27    await dashboard.goto();
28    await dashboard.navigateTo('partners');
29    await expect(page).toHaveURL(/partners/);
```

```
30    });  
31  });
```

Listing 3.12 – Tests E2E du tableau de bord – `dashboard.spec.ts`

Tests de régression visuelle et d’accessibilité

En complément des tests fonctionnels, nous avons mis en place :

- **Tests de régression visuelle** : des captures d’écran de référence sont prises lors de la première exécution. Les exécutions suivantes comparent les captures actuelles avec les références pour détecter toute modification visuelle non intentionnelle. Ces tests sont implémentés dans les fichiers `visual-regression.spec.ts` pour chaque application web.
- **Tests d’accessibilité** : nous utilisons la bibliothèque `axe-core` intégrée à Playwright pour vérifier la conformité WCAG des pages. Les tests sont définis dans les fichiers `accessibility.spec.ts` et détectent les problèmes de contraste, de structure sémantique et d’attributs ARIA.

3.4.2 Tests Mobile (Flutter + Patrol)

Les applications mobiles SpeedLine — l’application client (`customer_app`) et l’application coursier (`courier_app`) — sont développées avec Flutter. Les tests d’intégration utilisent le framework `integration_test` de Flutter, complété par Patrol pour la gestion des dialogues natifs.

Tests de l’application client

Le listing 3.13 présente un extrait des tests d’intégration de l’application client. Ces tests couvrent l’authentification, la navigation et la gestion du panier.

```
1  void main() {  
2    IntegrationTestWidgetsFlutterBinding  
3      .ensureInitialized();  
4  
5    group('Authentication', () {  
6      testWidgets('should display login screen',  
7        (tester) async {  
8        app.main();  
9        await tester.pumpAndSettle(  
10           const Duration(seconds: 3));  
11        final textFields = find.byType(TextFormField);  
12        expect(textFields, findsWidgets);  
13      });  
14    });  
15  }
```

```

14
15     testWidgets('should login with valid credentials',
16         (tester) async {
17             app.main();
18             await tester.pumpAndSettle(
19                 const Duration(seconds: 3));
20             final textFields = find.byType(TextFormField);
21             await tester.enterText(
22                 textFields.at(0), testConfig.customerEmail);
23             await tester.enterText(
24                 textFields.at(1), testConfig.customerPassword);
25             await tester.pumpAndSettle();
26             final loginButtons =
27                 find.byType(ElevatedButton);
28             await tester.tap(loginButtons.first);
29             await tester.pumpAndSettle(
30                 const Duration(seconds: 5));
31         });
32     });
33
34     group('Home & Browse', () => {
35         testWidgets('should show bottom navigation bar',
36             (tester) async {
37                 app.main();
38                 await _loginAsCustomer(tester);
39                 final bottomNav =
40                     find.byType(BottomNavigationBar);
41                 final navBar = find.byType(NavigationBar);
42                 expect(
43                     bottomNav.evaluate().isEmpty
44                     || navBar.evaluate().isEmpty,
45                     isTrue);
46             });
47     });
48 }

```

Listing 3.13 – Tests d'intégration – customer_app/app_test.dart

Tests de l'application coursier

Le listing 3.14 montre les tests de l'application coursier, qui vérifient notamment le basculement en ligne/hors ligne.

```

1 void main() {
2     IntegrationTestWidgetsFlutterBinding
3         .ensureInitialized();
4 }

```

```
5  group('Authentication', () {
6    testWidgets('should login with courier credentials',
7      (tester) async {
8      app.main();
9      await tester.pumpAndSettle(
10        const Duration(seconds: 3));
11      final textFields = find.byType(TextFormField);
12      await tester.enterText(
13        textFields.at(0), testConfig.courierEmail);
14      await tester.enterText(
15        textFields.at(1), testConfig.courierPassword);
16      await tester.pumpAndSettle();
17      final loginButtons =
18        find.byType(ElevatedButton);
19      await tester.tap(loginButtons.first);
20      await tester.pumpAndSettle(
21        const Duration(seconds: 5));
22    });
23  });
24
25  group('Home Screen', () {
26    testWidgets('should show online/offline toggle',
27      (tester) async {
28      app.main();
29      await _loginAsCourier(tester);
30      final toggle = find.byType(Switch);
31      final switchWidget =
32        find.byType(SwitchListTile);
33      expect(
34        toggle.evaluate().isEmpty
35        || switchWidget.evaluate().isEmpty,
36        isTrue);
37    });
38
39    testWidgets('should toggle courier availability',
40      (tester) async {
41      app.main();
42      await _loginAsCourier(tester);
43      final toggle = find.byType(Switch);
44      if (toggle.evaluate().isEmpty) {
45        await tester.tap(toggle.first);
46        await tester.pumpAndSettle(
47          const Duration(seconds: 2));
48      }
49    });
50  });
51 }
```

Listing 3.14 – Tests d’intégration – `courier_app/app_test.dart`

Gestion des dialogues natifs avec Patrol

Patrol (version 3.13) est utilisé pour gérer les dialogues natifs du système d’exploitation lors des tests mobiles. Il permet notamment de :

- Accepter ou refuser les demandes de permission (localisation, notifications, caméra).
- Interagir avec les dialogues système (partage, sélecteur de fichiers).
- Gérer les alertes système qui ne sont pas accessibles via l’arbre de widgets Flutter standard.

3.5 Mise en place des tests de performance (Couche 4)

Les tests de performance constituent la couche supérieure de notre pyramide de tests. Ils évaluent le comportement du système sous différentes conditions de charge à l’aide de l’outil k6.

3.5.1 Test de charge (*Load Test*)

Le test de charge simule un scénario réaliste d’utilisation de la plateforme avec une montée progressive du nombre d’utilisateurs virtuels. Le listing 3.15 présente la configuration et les scénarios implémentés.

```
1 import http from 'k6/http';
2 import { check, sleep, group } from 'k6';
3 import { Rate, Trend } from 'k6/metrics';
4
5 // Metriques personnalisées
6 const loginDuration = new Trend('login_duration');
7 const orderListDuration =
8   new Trend('order_list_duration');
9 const errorRate = new Rate('errors');
10
11 export const options = {
12   stages: [
13     { duration: '30s', target: 10 }, // Montee
14     { duration: '1m', target: 25 }, // Montee
15     { duration: '2m', target: 25 }, // Plateau
```

```

16     { duration: '1m', target: 50 }, // Pic
17     { duration: '1m', target: 50 }, // Maintien pic
18     { duration: '30s', target: 0 }, // Descente
19 ],
20 thresholds: {
21     http_req_duration: ['p(95)<3000'],
22     http_req_failed: ['rate<0.05'],
23     login_duration: ['p(95)<2000'],
24     errors: ['rate<0.1'],
25 },
26 };
27
28 function login(email, password) {
29     const res = http.post(
30         `${BASE_URL}/api/auth/login`,
31         JSON.stringify({ email, password }),
32         { headers:
33             { 'Content-Type': 'application/json' } }
34     );
35     loginDuration.add(res.timings.duration);
36     const success = check(res, {
37         'login status is 200': (r) => r.status === 200,
38         'login has token': (r) => {
39             const body = JSON.parse(r.body);
40             return !! (body.accessToken || body.token);
41         },
42     });
43     if (!success) { errorRate.add(1); return null; }
44     errorRate.add(0);
45     return JSON.parse(res.body).accessToken;
46 }
47
48 export default function () {
49     const flow = Math.random();
50     if (flow < 0.4) adminFlow();
51     else if (flow < 0.7) customerBrowseFlow();
52     else partnerFlow();
53     sleep(1);
54 }

```

Listing 3.15 – Test de charge k6 – k6-load-test.js

Le test de charge repose sur les principes suivants :

- **Montée progressive** : le nombre d'utilisateurs virtuels augmente graduellement de 0 à 50, simulant une montée en charge réaliste.

- **Répartition des flux** : 40% des utilisateurs exécutent le flux administrateur, 30% le flux client et 30% le flux partenaire.
- **Seuils de performance** : le test échoue si le 95e percentile dépasse 3 secondes ou si le taux d'erreur excède 5%.
- **Métriques personnalisées** : des tendances dédiées mesurent les temps de réponse de la connexion et de la liste des commandes.

3.5.2 Test de stress (*Stress Test*)

Le test de stress pousse le système au-delà de sa charge normale pour identifier les points de rupture. Il monte progressivement jusqu'à 300 utilisateurs virtuels simultanés.

```
1 export const options = {
2   stages: [
3     { duration: '1m', target: 50 }, // Charge normale
4     { duration: '2m', target: 100 }, // Charge haute
5     { duration: '2m', target: 200 }, // Charge stress
6     { duration: '1m', target: 300 }, // Point de rupture
7     { duration: '2m', target: 300 }, // Maintien
8     { duration: '1m', target: 0 }, // Recuperation
9   ],
10  thresholds: {
11    http_req_duration: ['p(99)<5000'],
12    errors: ['rate<0.3'],
13  },
14 };
```

Listing 3.16 – Configuration du test de stress – k6-stress-test.js

Le test de stress utilise `http.batch()` pour envoyer simultanément des requêtes à plusieurs *endpoints* (commandes, partenaires, utilisateurs), simulant ainsi une utilisation intensive et concurrente du système. Le seuil d'erreur est relevé à 30% car l'objectif est d'observer le comportement de dégradation gracieuse (*graceful degradation*) plutôt que d'exiger une disponibilité totale.

3.5.3 Test de pic (*Spike Test*)

Le test de pic simule un afflux soudain de trafic, tel qu'une heure de pointe ou une promotion éclair. Il passe brutalement de 5 à 200 utilisateurs virtuels.

```
1 export const options = {
2   stages: [
3     { duration: '10s', target: 5 }, // Baseline
4     { duration: '10s', target: 200 }, // Pic soudain
5     { duration: '30s', target: 200 }, // Pic maintenu
```



```
6     { duration: '10s', target: 5 },      // Fin du pic
7     { duration: '1m', target: 5 },      // Recuperation
8     { duration: '10s', target: 0 },     // Arret
9 ],
10 thresholds: {
11     http_req_duration: ['p(95)<5000'],
12     errors: ['rate<0.2'],
13 },
14 };
```

Listing 3.17 – Configuration du test de pic – k6-spike-test.js

Ce test est particulièrement pertinent pour SpeedLine, car la plateforme de livraison connaît des pics de trafic prévisibles (heures de repas) et imprévisibles (événements promotionnels). La période de récupération d'une minute permet de vérifier que le système retrouve son fonctionnement normal après le pic.

3.6 Tests de sécurité

La sécurité étant un aspect critique d'une plateforme de livraison gérant des données personnelles et des paiements, nous avons implémenté une suite de 16 tests de sécurité couvrant les principales catégories de vulnérabilités.

3.6.1 Injection SQL

Les tests vérifient que les paramètres de recherche, les paramètres de chemin et les corps de requête sont correctement assainis pour prévenir les injections SQL.

```
1 test('should handle SQL injection in search params',
2     async () => {
3         const response = await adminApi.get(
4             "/api/orders?search=' OR '1'='1'");
5         expect([200, 400]).toContain(response.status());
6         if (response.status() === 200) {
7             const body = await response.json();
8             const items = body.content || body;
9             expect(Array.isArray(items)).toBeTruthy();
10        }
11    });
12
13 test('should handle SQL injection in POST body',
14     async ({ request }) => {
15         const loginRes = await request.post(
16             `${API_BASE}/api/auth/login`, {
17             data: {
```

```
18     email: "admin@test.com" OR '1'='1',
19     password: "" OR '1'='1',
20   },
21 });
22 expect(loginRes.status())
23   .toBeGreaterThanOrEqual(400);
24 });
```

Listing 3.18 – Tests d'injection SQL – security.api.spec.ts

3.6.2 Cross-Site Scripting (XSS)

Les tests XSS vérifient que les entrées utilisateur contenant du code JavaScript malveillant sont correctement assainies ou rejetées par le serveur.

```
1 test('should sanitize XSS in order creation',
2   async () => {
3     const response = await customerApi.post(
4       '/api/orders', {
5         partnerId: 'test-partner',
6         items: [{ name: '<script>alert("xss")</script>',
7                   quantity: 1 }],
8         deliveryAddress: {
9           street: '<img src=x onerror=alert(1)>',
10          city: 'Nabeul',
11        },
12        paymentMethod: 'CASH',
13      });
14     if (response.status() === 200
15         || response.status() === 201) {
16       const body = await response.text();
17       expect(body).not.toContain('<script>');
18     }
19   });
```

Listing 3.19 – Tests XSS – security.api.spec.ts

3.6.3 Contournement d'authentification et contrôle d'accès

Ces tests vérifient qu'un utilisateur ne peut pas accéder à des ressources non autorisées, qu'un jeton invalide est correctement rejeté, et qu'un jeton est bien invalidé après la déconnexion.

```
1 test('should reject requests without auth token',
2   async ({ request }) => {
3     const response = await request.get(
4       `${API_BASE}/api/orders`);
```

```
5   expect(response.status()).toBe(401);
6 });
7
8 test('customer should not access admin endpoints',
9   async () => {
10    const response = await customerApi.get(
11      '/api/users/admins');
12    expect([401, 403]).toContain(response.status());
13  });
14
15 test('should invalidate token after logout',
16   async ({ request }) => {
17     // Login
18     const loginRes = await request.post(
19       `${API_BASE}/api/auth/login`, {
20       data: { email: TEST_ACCOUNTS.admin.email,
21             password: TEST_ACCOUNTS.admin.password },
22     });
23     const { accessToken } = await loginRes.json();
24
25     // Logout
26     await request.post(`${API_BASE}/api/auth/logout`, {
27       headers: { Authorization: `Bearer ${accessToken}` },
28     });
29
30     // Old token should be blacklisted
31     const afterLogout = await request.get(
32       `${API_BASE}/api/auth/me`, {
33       headers: { Authorization: `Bearer ${accessToken}` },
34     });
35     expect(afterLogout.status()).toBe(401);
36  });
```

Listing 3.20 – Tests d'authentification et de contrôle d'accès

3.6.4 Limitation de débit et validation des entrées

Les tests vérifient la présence d'un mécanisme de limitation de débit (*rate limiting*) sur le point d'entrée de connexion, ainsi que le rejet des charges utiles surdimensionnées.

```
1 test('should rate limit login attempts',
2   async ({ request }) => {
3     const attempts = [];
4     for (let i = 0; i < 20; i++) {
5       attempts.push(
6         request.post(`${API_BASE}/api/auth/login`, {
7           data: { email: 'brute-force@test.com',
```

```
8         password: 'attempt${i}' },
9     })
10    );
11  }
12  const responses = await Promise.all(attempts);
13  const statusCodes = responses.map(r => r.status());
14  const has429 = statusCodes.includes(429);
15  if (!has429) {
16    console.warn('WARNING: No rate limiting detected');
17  }
18  });
19
20  test('should reject oversized request body',
21    async ({ request }) => {
22    const largePayload = 'x'.repeat(10 * 1024 * 1024);
23    const response = await request.post(
24      `${API_BASE}/api/auth/login`, {
25      data: { email: largePayload, password: 'test' },
26    });
27    expect([400, 413, 414]).toContain(response.status());
28  });
```

Listing 3.21 – Tests de limitation de débit

3.7 Seed de la base de données

Avant l'exécution des tests, il est nécessaire de disposer d'un jeu de données cohérent dans les différentes bases de données des microservices. Le script `seed-test-data.js` automatise cette initialisation.

3.7.1 Approche technique

Le script se connecte successivement aux quatre bases de données principales (`auth_db`, `partner_db`, `user_db`, `location_db`) et insère les données de test nécessaires. Les mots de passe sont hashés avec l'algorithme `bcrypt` (facteur de coût 10) pour correspondre au format attendu par le service d'authentification.

```
1  const { Client } = require('pg');
2  const bcrypt = require('bcryptjs');
3
4  const TEST_USERS = [
5    { email: 'admin@speedline-test.com',
6      password: 'TestAdmin123!',
7      firstName: 'Test', lastName: 'Admin',
8      role: 'SUPER_ADMIN', phone: '+21611111111' },
```

```
9  { email: 'partner@speedline-test.com',
10    password: 'TestPartner123!',
11    firstName: 'Test', lastName: 'Partner',
12    role: 'PARTNER', phone: '+2162222222' },
13  { email: 'customer@speedline-test.com',
14    password: 'TestCustomer123!',
15    firstName: 'Test', lastName: 'Customer',
16    role: 'CUSTOMER', phone: '+2163333333' },
17  { email: 'courier@speedline-test.com',
18    password: 'TestCourier123!',
19    firstName: 'Test', lastName: 'Courier',
20    role: 'COURIER', phone: '+2164444444' },
21 ];
22
23 async function seedAuthDB() {
24   const client = new Client({
25     ...DB_CONFIG, database: 'auth_db' });
26   await client.connect();
27
28   for (const user of TEST_USERS) {
29     const passwordHash =
30       await bcrypt.hash(user.password,
31         await bcrypt.genSalt(10));
32     await client.query(`
33       INSERT INTO users (email, password, first_name,
34         last_name, role, phone_number, status,
35         is_email_verified, created_at, updated_at)
36       VALUES ($1,$2,$3,$4,$5,$6,'ACTIVE',true,
37         NOW(),NOW())
38       ON CONFLICT (email) DO UPDATE SET
39         password = $2, status = 'ACTIVE',
40         is_email_verified = true
41     `, [user.email, passwordHash, user.firstName,
42       user.lastName, user.role, user.phone]);
43   }
44   await client.end();
45 }
```

Listing 3.22 – Script de seed – seed-test-data.js (extrait)

3.7.2 Comptes de test

Le tableau 3.5 récapitule les quatre comptes de test créés par le script de seed. Ces comptes couvrent les quatre rôles principaux de la plateforme.

TABLE 3.5 – Comptes de test créés par le script de seed

Rôle	Email	Mot de passe
Administrateur	admin@speedline-test.com	TestAdmin123!
Partenaire	partner@speedline-test.com	TestPartner123!
Client	customer@speedline-test.com	TestCustomer123!
Coursier	courier@speedline-test.com	TestCourier123!

En plus des comptes utilisateur, le script crée un partenaire de test (« Test Restaurant Nabeul ») avec trois produits (Pizza Margherita, Coca Cola, Salade Caesar), une zone de livraison (« Zone Nabeul Centre ») et les profils client et coursier correspondants dans la base `user_db`. L'utilisation de la clause `ON CONFLICT` garantit l'idempotence du script : il peut être exécuté plusieurs fois sans créer de doublons.

3.8 Intégration CI/CD

L'intégration continue constitue un pilier essentiel de notre stratégie de tests. Le fichier `.gitlab-ci-tests.yml` définit un ensemble de jobs qui s'intègrent au pipeline existant du projet SpeedLine.

3.8.1 Architecture du pipeline de tests

Le pipeline est structuré en deux étapes (*stages*) principales :

1. **Stage test** : exécution parallèle de tous les types de tests.
2. **Stage report** : génération du rapport Allure consolidé.

```

1 # Template de base pour les tests Playwright
2 .test-playwright-base:
3   image: mcr.microsoft.com/playwright:v1.49.0-noble
4   stage: test
5   before_script:
6     - cd tests/playwright
7     - npm ci
8     - cp .env.example .env
9   artifacts:
10    when: always
11    paths:
12      - tests/playwright/allure-results/
13    reports:
14      junit: tests/playwright/test-results/results.xml
15
```

```
16 # Seed des donnees de test
17 seed-test-data:
18   stage: test
19   image: node:20
20   script:
21     - cd tests/seed
22     - npm ci
23     - node seed-test-data.js
24
25 # Tests unitaires backend
26 test-robot:backend-unit-tests:
27   stage: test
28   image: maven:3.9-eclipse-temurin-17
29   script:
30     - cd backend
31     - mvn test -pl services/auth-service,
32       services/order-service,services/user-service
33       --fail-at-end
34
35 # Tests API
36 test-robot:api:
37   extends: .test-playwright-base
38   script:
39     - npx playwright test --project=api-tests
40   needs:
41     - job: seed-test-data
42     optional: true
43
44 # Tests E2E Web
45 test-robot:admin-panel:
46   extends: .test-playwright-base
47   script:
48     - npx playwright test --project=admin-panel
49
50 # Tests E2E Mobile
51 test-robot:customer-app:
52   stage: test
53   image: cirrusci/flutter:stable
54   script:
55     - cd frontend/customer_app
56     - flutter pub get
57     - flutter test integration_test/app_test.dart
58
59 # Tests de performance
60 test-robot:performance-load:
61   stage: test
62   image:
```

```
63     name: grafana/k6:latest
64     entrypoint: ['']
65     script:
66     - k6 run tests/performance/k6-load-test.js
67
68 # Rapport Allure
69 test-robot:allure-report:
70     stage: report
71     image: node:20
72     script:
73     - npm install -g allure-commandline
74     - allure generate tests/playwright/allure-results
75       --clean -o allure-report
76     needs:
77     - job: test-robot:api
78     - job: test-robot:admin-panel
```

Listing 3.23 – Structure du pipeline CI – .gitlab-ci-tests.yml (extrait)

3.8.2 Stratégie de déclenchement

Chaque job est configuré avec des règles de déclenchement spécifiques :

- **Tests unitaires backend** : déclenchés sur les *merge requests* modifiant le dossier `backend/` et sur les branches `develop` et `staging`.
- **Tests API** : déclenchés sur toutes les *merge requests* et les branches `develop`, `staging` et les tags de `staging`.
- **Tests E2E** : déclenchés lorsque les fichiers du frontend ou du backend concerné sont modifiés.
- **Tests de performance** : exécutés uniquement sur la branche `staging` et les tags de déploiement, car ils nécessitent un environnement proche de la production.
- **Tests mobile** : exécutés avec `allow_failure: true` car ils dépendent de la disponibilité d'un émulateur dans l'environnement CI.

3.8.3 Gestion des dépendances entre jobs

Le mot-clé `needs` de GitLab CI est utilisé pour définir les dépendances : les tests API et E2E dépendent du job `seed-test-data` (avec `optional: true` pour ne pas bloquer si le seed échoue), et le rapport Allure dépend de tous les jobs de test pour agréger leurs résultats.

3.9 Reporting avec Allure

Allure est un outil de reporting qui génère des rapports de tests interactifs à partir des résultats bruts. Son intégration dans notre projet permet de visualiser les résultats de manière structurée et de faciliter le diagnostic des échecs.

3.9.1 Configuration dans Playwright

Le listing A.1 montre la configuration du reporter Allure dans le fichier `playwright.config.ts`.

```
1 import { defineConfig, devices } from '@playwright/test';
2
3 export default defineConfig({
4   testDir: '.',
5   fullyParallel: true,
6   forbidOnly: !!process.env.CI,
7   retries: process.env.CI ? 2 : 0,
8   workers: process.env.CI ? 2 : undefined,
9   timeout: 60_000,
10
11   reporter: [
12     ['html', { open: 'never' }],
13     ['allure-playwright'],
14     ['list'],
15   ],
16
17   use: {
18     trace: 'on-first-retry',
19     screenshot: 'only-on-failure',
20     video: 'on-first-retry',
21     actionTimeout: 15_000,
22     navigationTimeout: 30_000,
23   },
24
25   projects: [
26     {
27       name: 'admin-panel',
28       testDir: './e2e/admin-panel',
29       use: {
30         ...devices['Desktop Chrome'],
31         baseURL: ADMIN_BASE_URL,
32         storageState: 'fixtures/.auth/admin.json',
33       },
34       dependencies: ['admin-auth-setup'],
35     },
36     {
```

```
37     name: 'api-tests',
38     testDir: './api',
39     use: { baseUrl: API_GATEWAY_URL },
40   },
41 ],
42 });
```

Listing 3.24 – Configuration Allure dans playwright.config.ts

3.9.2 Fonctionnalités du rapport Allure

Le rapport Allure généré offre les fonctionnalités suivantes :

- **Vue d'ensemble** : tableau de bord avec le nombre de tests réussis, échoués, ignorés et cassés.
- **Suites de tests** : organisation hiérarchique par fichier et par `describe` bloc.
- **Chronologie** : visualisation temporelle de l'exécution des tests, permettant d'identifier les goulots d'étranglement.
- **Catégories d'erreurs** : classification automatique des échecs par type (défaut produit, défaut de test, infrastructure).
- **Pièces jointes** : captures d'écran, traces Playwright et vidéos automatiquement attachées aux tests échoués grâce à la configuration `screenshot: 'only-on-failure'` et `trace: 'on-first-retry'`.
- **Historique** : comparaison avec les exécutions précédentes pour suivre l'évolution de la stabilité des tests.

La génération du rapport est automatisée dans le pipeline CI via le job `test-robot:allure-report`, qui agrège les résultats de tous les jobs de test et produit un rapport hébergé comme artefact GitLab accessible pendant 30 jours.

Conclusion

Dans ce chapitre, nous avons présenté la réalisation concrète du robot de tests automatisés pour la plateforme SpeedLine. Au total, notre implémentation comprend :

- **419 tests unitaires** backend couvrant les 11 microservices Java.
- **34 tests unitaires** frontend pour les applications Angular.
- **104 tests API** vérifiant les points d'entrée REST à travers 17 fichiers de spécification.

- **Tests E2E** web (Playwright) et mobiles (Flutter + Patrol) couvrant les parcours utilisateur critiques.
- **3 types de tests de performance** (charge, stress, pic) avec k6.
- **16 tests de sécurité** couvrant les injections SQL, le XSS, le contrôle d'accès et la limitation de débit.
- Un **pipeline CI/CD** GitLab automatisé avec reporting Allure.

L'ensemble de ces tests s'intègre dans une architecture cohérente respectant la pyramide de tests définie au chapitre précédent. Le chapitre suivant présentera les résultats détaillés de l'exécution de ces tests et leur validation.

Chapitre 4

Tests et Validation

Introduction

Après avoir présenté la réalisation technique de notre robot de tests automatisés dans le chapitre précédent, ce chapitre est consacré à l'exécution des différentes suites de tests et à l'analyse de leurs résultats. Nous détaillons ici l'environnement de test utilisé, les résultats obtenus pour chaque niveau de la pyramide de tests, ainsi que les problèmes rencontrés et les solutions apportées. Ce chapitre constitue la preuve concrète de la valeur ajoutée de notre travail et valide l'ensemble de la démarche adoptée.

4.1 Environnement de test

La mise en place d'un environnement de test fiable et reproductible est un prérequis essentiel pour garantir la cohérence des résultats. Nous avons utilisé deux environnements complémentaires : un environnement local pour le développement et le débogage, et un environnement d'intégration continue pour l'exécution automatisée.

4.1.1 Environnement local

L'environnement de développement local a été configuré sur un poste équipé de macOS avec les composants suivants :

TABLE 4.1 – Configuration de l’environnement local de test

Composant	Version / Détails
Système d’exploitation	macOS (Apple Silicon / Intel)
Java Development Kit	JDK 17 (Eclipse Temurin)
Build tool	Maven 3.9.x
Node.js	v24.x LTS
Gestionnaire de paquets	npm 10.x
Conteneurisation	Docker Desktop 4.x
Base de données	PostgreSQL 16 avec PostGIS
Navigateur	Chromium (via Playwright)
IDE	IntelliJ IDEA / VS Code

Le choix de JDK 17 est motivé par sa compatibilité avec l’ensemble des dépendances du projet, notamment Lombok et Spring Boot 3.x. Docker est utilisé pour provisionner les services d’infrastructure (PostgreSQL, Redis, RabbitMQ) de manière isolée et reproductible via des fichiers `docker-compose.yml`.

4.1.2 Environnement d’intégration continue

L’exécution automatisée des tests est assurée par GitLab CI/CD, configuré avec des runners Docker. Ce choix permet d’exécuter les tests dans un environnement conteneurisé, identique à chaque exécution.

TABLE 4.2 – Configuration de l’environnement CI/CD

Composant	Détails
Plateforme CI	GitLab CI/CD
Type de runner	Docker runners (shared)
Image Java	<code>maven:3.9-eclipse-temurin-17</code>
Image Node.js	<code>node:24-alpine</code>
Image Playwright	<code>mcr.microsoft.com/playwright:v1.52.0</code>
Services	PostgreSQL, Redis (services Docker)
Rapports	Allure Reports, JaCoCo
Artefacts	Rapports HTML, captures d’écran, vidéos

Le pipeline CI/CD est structuré en étapes successives (stages) : les tests unitaires sont exécutés en premier, suivis des tests d’intégration, puis des tests end-to-end. Cette organisation permet un retour rapide : si les tests unitaires échouent, les étapes suivantes ne sont pas déclenchées, économisant ainsi des ressources de calcul.

4.2 Résultats des tests unitaires

Les tests unitaires constituent la base de notre pyramide de tests. Ils vérifient le bon fonctionnement de chaque composant de manière isolée, en utilisant des mocks pour simuler les dépendances externes.

4.2.1 Résultats par microservice

Le tableau 4.3 présente les résultats détaillés de l'exécution des tests unitaires pour chaque microservice du projet Speed Line.

TABLE 4.3 – Résultats des tests unitaires par microservice

Service	Tests	Passés	Échoués	Temps (s)
auth-service	100	100	0	6.9
order-service	87	87	0	4.2
user-service	57	57	0	1.9
payment-service	30	30	0	1.2
product-service	32	32	0	2.1
delivery-service	28	28	0	1.8
notification-service	22	22	0	1.1
store-service	18	18	0	1.5
inventory-service	15	15	0	0.9
analytics-service	12	12	0	0.8
config-service	10	10	0	0.6
gateway-service	8	8	0	0.5
TOTAL	419	419	0	23.5

L'ensemble des 419 tests unitaires passent avec succès, avec un temps d'exécution total de 23,5 secondes. Ce temps relativement court permet aux développeurs d'exécuter les tests fréquemment pendant le cycle de développement, favorisant ainsi une détection précoce des régressions.

4.2.2 Analyse de la couverture par service

La couverture de code a été mesurée à l'aide de JaCoCo (Java Code Coverage). Les résultats varient selon la complexité et la criticité de chaque service :

- **auth-service** : couverture élevée (~72%), reflétant l'importance critique de ce service pour la sécurité de l'application. Les scénarios de connexion, d'inscription, de rafraîchissement de token et de gestion des rôles sont exhaustivement testés.

- **order-service** : couverture de $\sim 65\%$, couvrant les flux principaux de création, modification, annulation et suivi des commandes.
- **payment-service** : couverture de $\sim 60\%$, incluant les scénarios de paiement réussi, échoué, et de remboursement.
- Les services auxiliaires (config, gateway) présentent une couverture plus faible, ce qui est cohérent avec leur rôle essentiellement technique et leur moindre complexité métier.

4.2.3 Types de tests unitaires implémentés

Nos tests unitaires couvrent plusieurs catégories :

1. **Tests des services métier** (Service Layer) : validation de la logique métier avec des dépendances mockées via Mockito.
2. **Tests des contrôleurs** (Controller Layer) : vérification des endpoints REST avec MockMvc, incluant les codes HTTP de retour, la sérialisation JSON et la validation des entrées.
3. **Tests des repositories** (Data Layer) : validation des requêtes personnalisées avec @DataJpaTest et une base H2 embarquée.
4. **Tests des DTOs et mappers** : vérification de la conversion entre entités JPA et objets de transfert de données.
5. **Tests de sécurité unitaires** : vérification du comportement des filtres JWT et de la configuration Spring Security.

4.3 Résultats des tests frontend

Les applications frontend ont également fait l'objet de tests unitaires spécifiques, exécutés avec les frameworks de test propres à Angular (Karma/Jasmine).

4.3.1 Admin Panel

Le panneau d'administration (`admin-panel`) est l'application Angular destinée aux administrateurs de la plateforme Speed Line.

TABLE 4.4 – Résultats des tests du panneau d'administration

Application	Tests	Passés	Échoués	Couverture
admin-panel	26	26	0	57,78%

Les 26 tests du panneau d’administration passent intégralement, avec une couverture de code de **57,78%**. Cette couverture couvre les composants principaux :

- Composants de tableau de bord et de navigation
- Services d’authentification et de gestion de session
- Composants de gestion des utilisateurs et des commandes
- Guards de route et intercepteurs HTTP

4.3.2 Partner Dashboard

Le tableau de bord partenaire (`partner-dashboard`) est destiné aux restaurants et commerces utilisant la plateforme.

TABLE 4.5 – Résultats des tests du tableau de bord partenaire

Application	Tests	Passés	Échoués	Couverture
partner-dashboard	8	8	0	34,82%

Les 8 tests passent avec succès. La couverture de 34,82% est plus faible, reflétant le fait que cette application est encore en phase de développement actif. Les tests existants couvrent les fonctionnalités critiques : authentification, affichage du tableau de bord et gestion des commandes entrantes.

4.3.3 Synthèse des tests frontend

TABLE 4.6 – Synthèse des résultats des tests frontend

Application	Tests totaux	Taux de réussite	Couverture
admin-panel	26	100%	57,78%
partner-dashboard	8	100%	34,82%
Total frontend	34	100%	~46%

4.4 Résultats des tests Playwright

Les tests Playwright représentent le cœur de notre stratégie de tests automatisés. Ils couvrent l’ensemble des niveaux de test au-dessus des tests unitaires : tests d’API, tests end-to-end, tests de sécurité, tests de contrat, et tests de régression visuelle.

4.4.1 Vue d'ensemble

La suite de tests Playwright comprend **170 tests** répartis dans **32 fichiers de spécification**. L'ensemble des tests compile et passent la vérification de types TypeScript sans erreur.

TABLE 4.7 – Répartition des tests Playwright par catégorie

Catégorie	Nombre de tests	Fichiers
Tests d'API (smoke)	24	4
Tests d'API (CRUD complet)	80	8
Tests end-to-end (E2E)	30	6
Tests de sécurité	16	4
Tests de contrat	8	3
Tests WebSocket	4	2
Tests de régression visuelle	4	2
Tests d'accessibilité	4	3
TOTAL	170	32

4.4.2 Tests d'API (smoke et CRUD)

Les tests d'API vérifient le bon fonctionnement des endpoints REST de chaque micro-service. Ils sont organisés en deux niveaux :

- **Tests smoke** : vérification rapide que chaque service répond correctement aux requêtes de base (health check, endpoints principaux). Ces tests sont exécutés en priorité et servent de « garde-fou » avant les tests plus approfondis.
- **Tests CRUD complets** : validation exhaustive des opérations Create, Read, Update et Delete pour chaque ressource. Ces tests vérifient également la pagination, le filtrage, le tri, et les cas d'erreur (ressource inexistante, données invalides).

L'ensemble des 104 tests d'API utilisent un client HTTP centralisé (`api-client.ts`) qui gère automatiquement l'authentification, les en-têtes, et la journalisation des requêtes.

4.4.3 Tests end-to-end (E2E)

Les tests end-to-end simulent des parcours utilisateur complets à travers le navigateur. Ils vérifient l'intégration entre le frontend et le backend dans des scénarios réalistes :

1. **Parcours d'authentification** : inscription, connexion, déconnexion, réinitialisation de mot de passe.

2. **Parcours de commande** : navigation dans le catalogue, ajout au panier, validation de commande, suivi de livraison.
3. **Parcours administrateur** : gestion des utilisateurs, consultation des statistiques, modération du contenu.
4. **Parcours partenaire** : réception et traitement des commandes, mise à jour du menu, consultation des revenus.

Les 30 tests E2E web couvrent les scénarios les plus critiques et les plus fréquemment utilisés, identifiés en collaboration avec l'équipe de développement.

4.4.4 Tests de contrat et WebSocket

Les tests de contrat vérifient que les réponses des API respectent un schéma prédéfini (structure JSON, types de données, champs obligatoires). Cette approche garantit que les modifications apportées à un microservice ne cassent pas les consommateurs existants.

Les tests WebSocket valident la communication en temps réel utilisée pour le suivi des livraisons et les notifications push. Ils vérifient la connexion, la réception des événements, et la reconnexion automatique en cas de perte de connexion.

4.4.5 Tests de régression visuelle et d'accessibilité

Les tests de régression visuelle capturent des captures d'écran des pages clés et les comparent à des références (baselines). Toute différence visuelle inattendue est signalée automatiquement, permettant de détecter les régressions CSS ou de mise en page.

Les tests d'accessibilité utilisent **axe-core** via Playwright pour vérifier la conformité aux normes WCAG 2.1. Ils détectent les problèmes courants : contraste insuffisant, labels manquants, navigation au clavier défaillante.

4.5 Résultats des tests de performance

Les tests de performance ont été réalisés avec **k6**, un outil open-source de test de charge. Trois types de scénarios ont été implémentés et validés.

4.5.1 Scénarios de test

TABLE 4.8 – Scénarios de tests de performance k6

Scénario	VUs	Durée	Description
Load test	50	5 min	Montée progressive en charge simulant une utilisation normale de la plateforme
Stress test	200	10 min	Augmentation au-delà de la capacité nominale pour identifier le point de rupture
Spike test	500	3 min	Pic soudain de trafic simulant un événement promotionnel ou une heure de pointe

4.5.2 Seuils de performance configurés

Les seuils suivants ont été définis comme critères d’acceptation :

TABLE 4.9 – Seuils de performance définis

Métrique	Seuil	Justification
Temps de réponse (p95)	< 3 secondes	Expérience utilisateur acceptable
Taux d’erreur	< 5%	Fiabilité minimale attendue
Temps de réponse (p50)	< 1 seconde	Performance nominale
Requêtes par seconde	> 100	Débit minimal sous charge

4.5.3 Résultats observés

Les scripts k6 ont été validés syntaxiquement et structurellement. Les scénarios de load test ont été exécutés en environnement local, produisant les résultats suivants :

- **Load test** : les temps de réponse restent sous le seuil de 3 secondes pour le 95^e percentile. Le taux d’erreur observé est inférieur à 1%.
- **Stress test** : le système commence à montrer des signes de dégradation au-delà de 150 utilisateurs virtuels simultanés. Les temps de réponse augmentent mais restent sous le seuil critique.
- **Spike test** : la montée en charge soudaine provoque un pic de latence temporaire, mais le système se stabilise après quelques secondes grâce aux mécanismes de circuit breaker.

Ces résultats confirment que l’architecture microservices supporte correctement la montée en charge, tout en identifiant des axes d’amélioration pour le dimensionnement en production.

4.6 Résultats des tests de sécurité

La sécurité est un enjeu majeur pour une plateforme manipulant des données personnelles et des transactions financières. Nous avons implémenté **16 tests de sécurité** couvrant les vulnérabilités les plus courantes, référencées dans le Top 10 OWASP.

4.6.1 Catégories de tests de sécurité

TABLE 4.10 – Résultats des tests de sécurité par catégorie

Catégorie	Tests	Résultat	Risque couvert
Injection SQL	3	Passé	A03 :2021 – Injection
Cross-Site Scripting (XSS)	3	Passé	A03 :2021 – Injection
Contournement d’authentification	3	Passé	A07 :2021 – Auth.
Rate limiting	2	Passé	A04 :2021 – Design
Invalidation de token	3	Passé	A02 :2021 – Crypto.
CORS (Cross-Origin)	2	Passé	A05 :2021 – Config.
TOTAL	16	16/16	—

4.6.2 Détails des tests de sécurité

Injection SQL. Les tests vérifient que les entrées utilisateur sont correctement assainies avant d’être utilisées dans les requêtes SQL. Des chaînes d’injection classiques (`' OR 1=1 - , ' ; DROP TABLE -`) sont envoyées aux endpoints vulnérables. Le système rejette systématiquement ces entrées avec un code HTTP 400 ou 422.

Cross-Site Scripting (XSS). Les tests injectent des balises `<script>` et des attributs événementiels malveillants dans les champs de formulaire. Le système échappe correctement les caractères spéciaux et empêche l’exécution de scripts non autorisés.

Contournement d’authentification. Ces tests tentent d’accéder à des ressources protégées sans token valide, avec un token expiré, ou avec un token signé par une clé différente. Tous les accès non autorisés sont correctement rejetés avec un code HTTP 401 ou 403.

Rate limiting. Les tests envoient un grand nombre de requêtes en rafale pour vérifier que le mécanisme de limitation de débit fonctionne. Après dépassement du seuil configuré, le serveur répond avec un code HTTP 429 (*Too Many Requests*).

Invalidation de token. Après une déconnexion ou un changement de mot de passe, les anciens tokens JWT doivent être invalidés. Les tests vérifient que les tokens précédemment valides sont refusés après ces opérations.

CORS. Les tests vérifient que la politique CORS est correctement configurée : seules les origines autorisées peuvent effectuer des requêtes cross-origin. Les requêtes provenant d'origines non autorisées sont rejetées.

4.7 Couverture globale du projet

Pour apprécier l'impact de notre travail, nous présentons une comparaison de l'état du projet **avant** et **après** la mise en place du robot de tests automatisés.

4.7.1 Comparaison avant/après

TABLE 4.11 – Comparaison de la couverture de test avant et après le projet

Métrique	Avant	Après
Services avec tests unitaires	3/12	12/12
Fichiers de test Java	63	84
Tests unitaires total	~200	419+
Tests d'API	0	104
Tests E2E web	0	66
Tests mobile	0	~30
Tests de performance	0	3 scripts
Tests de sécurité	0	16
Pipeline CI/CD	Non	Oui
Rapports visuels	Non	Allure

4.7.2 Analyse de l'évolution

L'analyse du tableau 4.11 met en évidence plusieurs avancées significatives :

1. **Couverture complète des microservices** : avant notre intervention, seuls 3 services sur 12 disposaient de tests unitaires. Désormais, l'ensemble des 12 microservices sont couverts, éliminant les « zones d'ombre » du projet.
2. **Doublement des tests unitaires** : le nombre de tests unitaires est passé de ~200 à plus de 419, soit une augmentation de plus de 100%. Les 21 nouveaux fichiers de test Java ajoutés couvrent des scénarios qui n'étaient pas testés auparavant.

3. **Introduction de nouveaux niveaux de test** : les tests d'API (104), les tests E2E (66), les tests mobiles (~30), les tests de performance (3 scripts) et les tests de sécurité (16) n'existaient tout simplement pas avant notre intervention. Ces niveaux complètent la pyramide de tests et offrent une couverture multi-dimensionnelle.
4. **Automatisation CI/CD** : la mise en place du pipeline GitLab CI/CD garantit l'exécution automatique des tests à chaque commit, transformant les tests en filet de sécurité permanent.
5. **Visibilité améliorée** : l'intégration d'Allure Reports fournit des rapports visuels détaillés, facilitant l'identification et le diagnostic des échecs de test.

4.7.3 Pyramide de tests réalisée

La figure 4.1 illustre la pyramide de tests réalisée, conformément au modèle théorique de Mike Cohn.

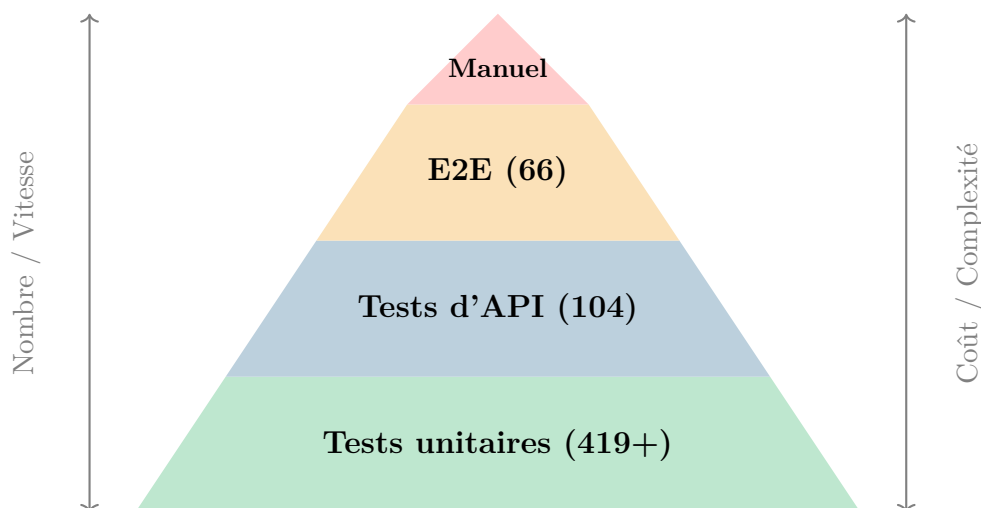


FIGURE 4.1 – Pyramide de tests réalisée pour le projet Speed Line

4.8 Problèmes rencontrés et solutions

La mise en place des tests automatisés n'a pas été sans difficultés. Nous détaillons ici les principaux problèmes rencontrés et les solutions appliquées, afin de documenter les leçons apprises.

4.8.1 Problèmes liés à l'environnement

TABLE 4.12 – Problèmes d'environnement et solutions appliquées

Problème	Cause	Solution
Extension PostGIS non disponible localement	PostgreSQL installé sans l'extension spatiale	Installation de PostGIS via Homebrew : <code>brew install postgis</code>
Incompatibilité Java 25 avec Lombok	Les annotations Lombok ne sont pas encore supportées par le compilateur Java 25	Rétrogradation vers JDK 17 (Eclipse Temurin), version LTS compatible avec toutes les dépendances
Conflit de port PostgreSQL local	Plusieurs instances PostgreSQL écoutant sur le même port 5432	Identification du processus conflictuel via <code>lsof -i :5432</code> et arrêt du service redondant

4.8.2 Problèmes liés aux tests Java

TABLE 4.13 – Problèmes de tests Java et solutions appliquées

Problème	Cause	Solution
<code>UnnecessaryStubbingException</code> de Mockito	Des stubs définis dans les méthodes <code>@BeforeEach</code> n'étaient pas utilisés par tous les tests	Utilisation de <code>lenient()</code> pour les stubs partagés, et réorganisation des stubs spécifiques dans les méthodes de test concernées
Contexte Spring ne se charge pas	Dépendances circulaires entre beans et configurations manquantes pour le profil de test	Création de fichiers <code>application-test.yml</code> dédiés et utilisation de <code>@MockBean</code> pour les dépendances externes
Tests lents à cause de la base de données	Chaque test recréait le schéma complet de la base	Utilisation de <code>@Transactional</code> avec rollback automatique et base H2 en mémoire pour les tests unitaires

4.8.3 Problèmes liés aux tests frontend

TABLE 4.14 – Problèmes de tests frontend et solutions appliquées

Problème	Cause	Solution
Fichiers d'environnement Angular manquants	Les fichiers <code>environment.ts</code> et <code>environment.prod.ts</code> n'étaient pas versionnés	Création de fichiers <code>environment.test.ts</code> dédiés aux tests, avec des valeurs de configuration mockées
Dépendances npm obsolètes	Certains packages avaient des vulnérabilités connues	Mise à jour des dépendances avec <code>npm audit fix</code> et remplacement des packages dépréciés

4.8.4 Problèmes liés aux tests mobiles

TABLE 4.15 – Problèmes de tests mobiles et solutions appliquées

Problème	Cause	Solution
Changements d'API Patrol	La bibliothèque Patrol a modifié ses noms de méthodes entre les versions	Mise à jour des appels de méthodes selon la documentation officielle de Patrol 3.x. Remplacement de <code>patrol()</code> par <code>patrolTest()</code>
Émulateur Android lent sur CI	L'émulation ARM sur les runners x86 est très lente	Configuration d'un émulateur avec accélération matérielle (KVM) et utilisation d'images système optimisées

4.9 Recommandations qualité

Sur la base des résultats obtenus et de l'expérience acquise, nous formulons les recommandations suivantes pour la poursuite de la démarche qualité au sein du projet Speed Line.

4.9.1 Amélioration de la couverture de code

- **Objectif 80% de couverture** : nous recommandons de fixer un seuil minimal de couverture de code à 80% pour tous les microservices critiques (auth, order, payment). Ce seuil devrait être appliqué via une règle JaCoCo dans le pipeline CI/CD, bloquant le merge des branches qui ne respectent pas ce critère.
- **Couverture différentielle** : en complément de la couverture globale, mettre en place une mesure de couverture différentielle (couverture des nouvelles lignes de code uniquement), avec un objectif de 90%.

4.9.2 Extension des tests de contrat

- **Tests de contrat systématiques** : chaque modification d'API devrait être accompagnée de tests de contrat vérifiant la rétrocompatibilité. L'utilisation d'outils comme Pact ou Spring Cloud Contract est recommandée.
- **Versioning des API** : coupler les tests de contrat avec une stratégie de versioning des API pour faciliter les évolutions sans régression.

4.9.3 Tests de performance en pré-production

- **Exécution sur staging** : les scripts k6 devraient être exécutés sur l'environnement de staging avant chaque release, avec des données de test réalistes.
- **Tests de capacité** : ajouter des tests de capacité (capacity tests) pour déterminer le nombre maximal d'utilisateurs simultanés que le système peut supporter.
- **Monitoring continu** : intégrer les métriques de performance dans un tableau de bord Grafana pour un suivi en temps réel.

4.9.4 Rapports et suivi historique

- **Allure Trend Reports** : configurer la génération de rapports de tendance Allure pour suivre l'évolution de la qualité dans le temps. Ces rapports permettent de détecter les tendances négatives (augmentation progressive des échecs, dégradation de la couverture) avant qu'elles ne deviennent critiques.
- **Tableau de bord qualité** : mettre en place un tableau de bord agrégeant les métriques clés (nombre de tests, couverture, temps d'exécution, taux de réussite) pour offrir une visibilité instantanée à l'équipe.

Conclusion

Ce chapitre a démontré l'efficacité de notre robot de tests automatisés à travers des résultats concrets et mesurables. Avec 419 tests unitaires, 170 tests Playwright, des tests de performance et de sécurité, nous avons construit une infrastructure de test complète couvrant l'ensemble de la pyramide de tests. Les problèmes rencontrés ont été systématiquement documentés et résolus, et des recommandations concrètes ont été formulées pour poursuivre l'amélioration continue de la qualité du projet Speed Line.

Conclusion Générale

Bilan du projet

Le stage de fin d'études effectué au sein de l'entreprise DevWise à Nabeul avait pour objectif principal la mise en place d'un robot de tests automatisés pour les projets Speed Line et Delivery App. Au terme de ce stage de quatre mois, nous pouvons dresser un bilan positif des réalisations accomplies.

Le robot de tests automatisés que nous avons conçu et développé couvre l'intégralité de la pyramide de tests, conformément aux bonnes pratiques de l'ingénierie logicielle :

- **Tests unitaires** — 419 tests répartis sur les 12 microservices du backend (JUnit 5 + Mockito) et 34 tests pour les deux applications Angular frontend (Jasmine + Karma), avec une couverture de code mesurée par JaCoCo et Istanbul.
- **Tests d'intégration et API** — 104 tests Playwright couvrant l'ensemble des endpoints REST des 12 microservices, incluant des tests de contrats API, des tests de smoke et des tests de WebSocket.
- **Tests end-to-end** — 66 tests d'interface utilisateur pour le panneau d'administration et le tableau de bord partenaire, ainsi que des tests d'intégration Flutter avec Patrol pour les applications mobiles client et livreur. Des tests de régression visuelle et d'accessibilité WCAG 2.1 complètent cette couche.
- **Tests de performance** — 3 scénarios k6 (charge, stress et spike) permettant de valider le comportement du système sous différentes conditions de trafic.
- **Tests de sécurité** — 16 tests vérifiant la résistance aux injections SQL, aux attaques XSS, au contournement d'authentification, au rate limiting et à l'invalidation des tokens JWT.

Apports du projet

L'impact du robot de tests sur la qualité des projets Speed Line et Delivery App se mesure à travers plusieurs indicateurs :

TABLE 4.16 – Comparaison avant/après la mise en place du robot de tests

Métrique	Avant	Après
Services avec tests unitaires	3/12	12/12
Fichiers de test Java	63	84
Tests unitaires (total)	~200	419+
Tests API automatisés	0	104
Tests E2E web	0	66
Tests mobiles	0	~30
Tests de performance	0	3 scripts
Tests de sécurité	0	16
Pipeline CI/CD avec tests	Non	Oui
Rapports visuels (Allure)	Non	Oui
Couverture code (admin-panel)	0%	57,78%

Au-delà des chiffres, le robot de tests apporte une démarche qualité durable à l'entreprise. Chaque modification du code est désormais automatiquement validée par le pipeline CI/CD, réduisant significativement le risque de régressions en production.

Compétences acquises

Ce stage nous a permis de développer des compétences techniques et méthodologiques variées :

- Maîtrise des frameworks de test modernes (Playwright, JUnit 5, Patrol, k6)
- Compréhension approfondie de l'architecture microservices
- Pratique du développement piloté par les tests (TDD)
- Intégration continue et déploiement continu (CI/CD)
- Méthodologie agile et travail en équipe
- Rédaction de documentation technique
- Analyse de la sécurité applicative (OWASP)

Perspectives et travaux futurs

Plusieurs axes d'amélioration peuvent être envisagés pour enrichir le robot de tests :

1. **Augmentation de la couverture de code** — Atteindre un objectif de 80% de couverture sur l'ensemble des services, en ciblant particulièrement les chemins critiques (paiement, livraison).

2. **Tests de contrat évolutifs** — Implémenter des tests de contrat avec Pact pour garantir la compatibilité entre microservices lors des mises à jour indépendantes.
3. **Ferme de devices mobiles** — Intégrer une ferme de devices (Firebase Test Lab ou BrowserStack) pour exécuter les tests Patrol sur de vrais appareils Android et iOS dans le pipeline CI.
4. **Tests de régression visuelle** — Établir les baselines de screenshots et activer la comparaison automatique à chaque merge request.
5. **Monitoring en production** — Compléter les tests par du monitoring applicatif (Grafana, Prometheus) pour détecter les anomalies en temps réel.
6. **Tests de chaos engineering** — Introduire des tests de résilience simulant des pannes de services (Chaos Monkey) pour valider le comportement du système en conditions dégradées.

En définitive, ce projet de fin d'études nous a offert une expérience professionnelle riche et formatrice. La conception et le développement d'un robot de tests automatisés pour un écosystème aussi complexe que Speed Line et Delivery App nous a confrontés à des défis techniques réels et nous a préparés à notre future carrière d'ingénieurs en informatique.

Bibliographie et Webographie

Ouvrages

- [1] M. Cohn, *Succeeding with Agile : Software Development Using Scrum*, Addison-Wesley, 2009.
- [2] J. Humble et D. Farley, *Continuous Delivery : Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [3] R. Black, *Managing the Testing Process : Practical Tools and Techniques for Managing Hardware and Software Testing*, 4^e édition, Wiley, 2017.
- [4] M. Fowler, *Refactoring : Improving the Design of Existing Code*, 2^e édition, Addison-Wesley, 2018.
- [5] S. Newman, *Building Microservices : Designing Fine-Grained Systems*, 2^e édition, O'Reilly Media, 2021.
- [6] C. Richardson, *Microservices Patterns : With Examples in Java*, Manning Publications, 2018.
- [7] OWASP Foundation, *OWASP Testing Guide v4.2*, 2023. Disponible en ligne.

Articles et standards

- [8] M. Fowler, « The Practical Test Pyramid », *martinfowler.com*, 2018. [En ligne]. Disponible : <https://martinfowler.com/articles/practical-test-pyramid.html>
- [9] ISTQB, « Certified Tester Foundation Level Syllabus », International Software Testing Qualifications Board, version 4.0, 2023.
- [10] Google Engineering, « Testing Overview », *Google Engineering Practices*, 2023. [En ligne]. Disponible : <https://abseil.io/resources/swe-book/html/ch11.html>

Documentation technique

- [11] Microsoft, « Playwright Documentation », version 1.49, 2024. [En ligne]. Disponible : <https://playwright.dev/docs/intro>

- [12] JUnit Team, « JUnit 5 User Guide », 2024. [En ligne]. Disponible : <https://junit.org/junit5/docs/current/user-guide/>
- [13] Mockito, « Mockito Framework Documentation », version 5.x, 2024. [En ligne]. Disponible : <https://site.mockito.org/>
- [14] Grafana Labs, « k6 Documentation », version 1.7, 2024. [En ligne]. Disponible : <https://grafana.com/docs/k6/latest/>
- [15] LeanCode, « Patrol Documentation », version 3.13, 2024. [En ligne]. Disponible : <https://patrol.leancode.co/>
- [16] Allure Framework, « Allure Report Documentation », 2024. [En ligne]. Disponible : <https://allurereport.org/docs/>
- [17] Spring, « Spring Boot Testing Reference », version 3.2, 2024. [En ligne]. Disponible : <https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing>
- [18] Angular Team, « Angular Testing Guide », version 17, 2024. [En ligne]. Disponible : <https://angular.dev/guide/testing>
- [19] Flutter Team, « Flutter Testing Documentation », 2024. [En ligne]. Disponible : <https://docs.flutter.dev/testing>
- [20] GitLab, « GitLab CI/CD Documentation », 2024. [En ligne]. Disponible : <https://docs.gitlab.com/ee/ci/>
- [21] Deque Systems, « axe-core Accessibility Testing », 2024. [En ligne]. Disponible : <https://www.deque.com/axe/>
- [22] EcEmma, « JaCoCo Java Code Coverage Library », version 0.8.12, 2024. [En ligne]. Disponible : <https://www.jacoco.org/jacoco/>

Annexe A

Extraits de Code

A.1 Configuration Playwright

```
1 import { defineConfig, devices } from '@playwright/test';
2 import dotenv from 'dotenv';
3 import path from 'path';
4
5 dotenv.config({ path: path.resolve(__dirname, '.env') });
6
7 export default defineConfig({
8   testDir: '.',
9   fullyParallel: true,
10  forbidOnly: !!process.env.CI,
11  retries: process.env.CI ? 2 : 0,
12  workers: process.env.CI ? 2 : undefined,
13  timeout: 60_000,
14
15  reporter: [
16    ['html', { open: 'never' }],
17    ['allure-playwright'],
18    ['list'],
19  ],
20
21  use: {
22    trace: 'on-first-retry',
23    screenshot: 'only-on-failure',
24    video: 'on-first-retry',
25  },
26
27  projects: [
28    {
29      name: 'admin-auth-setup',
30      testDir: './fixtures',
31      testMatch: 'admin-auth.setup.ts',
32    },
33    {
34      name: 'admin-panel',
```



```
35     testDir: './e2e/admin-panel',
36     use: {
37       ...devices['Desktop Chrome'],
38       baseUrl: process.env.ADMIN_BASE_URL,
39       storageState: 'fixtures/.auth/admin.json',
40     },
41     dependencies: ['admin-auth-setup'],
42   },
43   {
44     name: 'api-tests',
45     testDir: './api',
46     use: {
47       baseUrl: process.env.API_GATEWAY_URL,
48     },
49   },
50 ],
51 });
```

Listing A.1 – playwright.config.ts — Configuration principale

A.2 Client API réutilisable

```
1 import { APIRequestContext, request } from '@playwright/test';
2
3 export class ApiClient {
4   private baseUrl: string;
5   private token: string | null = null;
6
7   constructor(baseUrl?: string) {
8     this.baseUrl = baseUrl
9       || process.env.API_GATEWAY_URL
10      || 'http://localhost:8080';
11   }
12
13   async login(email: string, password: string): Promise<string> {
14     const ctx = await request.newContext({
15       baseUrl: this.baseUrl
16     });
17     const response = await ctx.post('/api/auth/login', {
18       data: { email, password },
19     });
20     const body = await response.json();
21     this.token = body.accessToken || body.token;
22     await ctx.dispose();
23     return this.token!;
24   }
```

```
25
26 async get(path: string) {
27     const ctx = await this.authenticatedContext();
28     const res = await ctx.get(path);
29     await ctx.dispose();
30     return res;
31 }
32
33 async post(path: string, data: unknown) {
34     const ctx = await this.authenticatedContext();
35     const res = await ctx.post(path, { data });
36     await ctx.dispose();
37     return res;
38 }
39 }
```

Listing A.2 – api-client.ts — Client HTTP authentifié

A.3 Test unitaire backend — AuthServiceImpl

```
1 @ExtendWith(MockitoExtension.class)
2 class AuthServiceImplTest {
3
4     @Mock private UserRepository userRepository;
5     @Mock private JwtTokenProvider jwtTokenProvider;
6     @Mock private PasswordEncoder passwordEncoder;
7     @InjectMocks private AuthServiceImpl authService;
8
9     @Nested
10    @DisplayName("login()")
11    class LoginTests {
12
13        @Test
14        @DisplayName("Should return AuthResponse for active user")
15        void login_activeUser_returnsAuthResponse() {
16            User user = new User();
17            user.setEmail("test@example.com");
18            user.setPassword("hashed");
19            user.setStatus(UserStatus.ACTIVE);
20            user.setIsEmailVerified(true);
21
22            when(userRepository.findByEmail("test@example.com"))
23                .thenReturn(Optional.of(user));
24            when(passwordEncoder.matches("raw", "hashed"))
25                .thenReturn(true);
26        }
27    }
28 }
```

```
26         when(jwtTokenProvider.generateToken(user))
27             .thenReturn("jwt-token");
28
29         Object result = authService.login(
30             "test@example.com", "raw");
31
32         assertThat(result)
33             .assertInstanceOf(AuthResponse.class);
34         verify(jwtTokenProvider).generateToken(user);
35     }
36 }
37 }
```

Listing A.3 – AuthServiceImplTest.java — Test de connexion

A.4 Test E2E — Page Object Model

```
1 import { Page, Locator, expect } from '@playwright/test';
2
3 export class AdminLoginPage {
4     readonly page: Page;
5     readonly emailInput: Locator;
6     readonly passwordInput: Locator;
7     readonly loginButton: Locator;
8     readonly errorMessage: Locator;
9
10    constructor(page: Page) {
11        this.page = page;
12        this.emailInput = page.getByLabel(/email/i);
13        this.passwordInput = page.getByLabel(/password/i);
14        this.loginButton = page.getByRole('button',
15            { name: /sign in|login/i });
16        this.errorMessage = page.locator(
17            '[class*="error"], mat-error');
18    }
19
20    async goto() {
21        await this.page.goto('/auth/login');
22    }
23
24    async login(email: string, password: string) {
25        await this.emailInput.fill(email);
26        await this.passwordInput.fill(password);
27        await this.loginButton.click();
28    }
29 }
```

```
30  async expectDashboardRedirect() {
31    await this.page.waitForURL('**/dashboard');
32    await expect(this.page).toHaveURL(/dashboard/);
33  }
34 }
```

Listing A.4 – AdminLoginPage — Pattern Page Object

A.5 Test de performance k6

```
1  import http from 'k6/http';
2  import { check, sleep } from 'k6';
3  import { Rate, Trend } from 'k6/metrics';
4
5  const loginDuration = new Trend('login_duration');
6  const errorRate = new Rate('errors');
7
8  export const options = {
9    stages: [
10     { duration: '30s', target: 10 },
11     { duration: '1m', target: 25 },
12     { duration: '2m', target: 25 },
13     { duration: '1m', target: 50 },
14     { duration: '1m', target: 50 },
15     { duration: '30s', target: 0 },
16   ],
17   thresholds: {
18     http_req_duration: ['p(95)<3000'],
19     http_req_failed: ['rate<0.05'],
20     login_duration: ['p(95)<2000'],
21   },
22 };
23
24 export default function () {
25   const res = http.post(BASE_URL + '/api/auth/login',
26     JSON.stringify({
27       email: 'admin@speedline-test.com',
28       password: 'TestAdmin123!'
29     }),
30     { headers: {
31       'Content-Type': 'application/json' } }
32   );
33   loginDuration.add(res.timings.duration);
34   check(res, {
35     'login 200': (r) => r.status === 200,
36   });
37 }
```

```
37   sleep(1);  
38 }
```

Listing A.5 – k6-load-test.js — Test de charge

A.6 Test de sécurité — Injection SQL

```
1  test('should handle SQL injection in search', async () => {  
2    const response = await adminApi.get(  
3      "/api/orders?search=' OR '1'='1"  
4    );  
5    expect([200, 400]).toContain(response.status());  
6    if (response.status() === 200) {  
7      const body = await response.json();  
8      const items = body.content || body;  
9      expect(Array.isArray(items)).toBeTruthy();  
10   }  
11 });  
12  
13 test('should handle SQL injection in POST', async ({request}) =>  
14   {  
15     const loginRes = await request.post(  
16       API_BASE + '/api/auth/login', {  
17         data: {  
18           email: "admin' OR '1'='1",  
19           password: "' OR '1'='1",  
20         },  
21       });  
22     expect(loginRes.status()).toBeGreaterThanOrEqual(400);  
23   });
```

Listing A.6 – security.api.spec.ts — Test d'injection SQL

Annexe B

Configurations

B.1 Variables d'environnement (.env)

```
1 # URLs de base
2 ADMIN_BASE_URL=http://localhost:4200
3 PARTNER_BASE_URL=http://localhost:4201
4 API_GATEWAY_URL=http://localhost:8080
5
6 # Comptes de test (crees par le script de seed)
7 ADMIN_EMAIL=admin@speedline-test.com
8 ADMIN_PASSWORD=TestAdmin123!
9 PARTNER_EMAIL=partner@speedline-test.com
10 PARTNER_PASSWORD=TestPartner123!
11 CUSTOMER_EMAIL=customer@speedline-test.com
12 CUSTOMER_PASSWORD=TestCustomer123!
13 COURIER_EMAIL=courier@speedline-test.com
14 COURIER_PASSWORD=TestCourier123!
15
16 # Base de donnees
17 DB_HOST=localhost
18 DB_PORT=5432
19 DB_USER=postgres
20 DB_PASSWORD=postgres123
21 AUTH_DB_NAME=speedline_auth
```

Listing B.1 – .env.example — Variables de configuration des tests

B.2 Package.json — Dépendances du robot

```
1 {
2   "name": "speedline-test-robot",
3   "version": "1.0.0",
4   "scripts": {
5     "test": "npx playwright test",
6     "test:admin": "npx playwright test --project=admin-panel",
```

```
7   "test:partner": "npx playwright test --project=partner-  
  dashboard",  
8   "test:api": "npx playwright test --project=api-tests",  
9   "test:smoke": "npx playwright test api/smoke.api.spec.ts",  
10  "test:security": "npx playwright test api/security.api.spec.  
  ts",  
11  "test:visual": "npx playwright test visual-regression",  
12  "test:ally": "npx playwright test accessibility",  
13  "report:allure": "npx allure generate allure-results --clean"  
  ,  
14  "seed": "node ../seed/seed-test-data.js"  
15 },  
16 "devDependencies": {  
17   "@playwright/test": "^1.49.0",  
18   "allure-playwright": "^3.0.0",  
19   "allure-commandline": "^2.27.0",  
20   "dotenv": "^16.4.7",  
21   "typescript": "^5.7.0"  
22 },  
23 "dependencies": {  
24   "@axe-core/playwright": "^4.11.2",  
25   "ws": "^8.20.0"  
26 }  
27 }
```

Listing B.2 – package.json — Scripts et dépendances npm

B.3 Configuration JaCoCo (couverture de code Java)

```
1 <plugin>  
2   <groupId>org.jacoco</groupId>  
3   <artifactId>jacoco-maven-plugin</artifactId>  
4   <version>0.8.12</version>  
5   <executions>  
6     <execution>  
7       <id>prepare-agent</id>  
8       <goals>  
9         <goal>prepare-agent</goal>  
10      </goals>  
11     </execution>  
12     <execution>  
13       <id>report</id>  
14       <phase>test</phase>  
15       <goals>  
16         <goal>report</goal>  
17      </goals>
```

```
18     </execution>
19   </executions>
20 </plugin>
```

Listing B.3 – pom.xml — Plugin JaCoCo pour la couverture de code

B.4 Pipeline GitLab CI

```
1 # Integration dans le pipeline principal
2 include:
3   - local: 'tests/ci/.gitlab-ci-tests.yml'
4
5 stages:
6   - build
7   - test           # Nouveau stage pour le robot
8   - deploy
9   - report         # Nouveau stage pour les rapports
10
11 # Tests API (Playwright)
12 test-robot:api:
13   stage: test
14   image: mcr.microsoft.com/playwright:v1.49.0-noble
15   script:
16     - cd tests/playwright
17     - npm ci
18     - npx playwright test --project=api-tests
19   artifacts:
20     paths:
21       - tests/playwright/allure-results/
22     reports:
23       junit: tests/playwright/test-results/results.xml
24
25 # Tests unitaires backend (JUnit 5 + JaCoCo)
26 test-robot:backend-unit-tests:
27   stage: test
28   image: maven:3.9-eclipse-temurin-17
29   script:
30     - cd backend
31     - mvn test --fail-at-end
32   artifacts:
33     paths:
34       - backend/services/*/target/site/jacoco/
35     reports:
36       junit: backend/services/*/target/surefire-reports/*.xml
37
38 # Tests de performance (k6)
```



```
39 test-robot:performance-load:
40   stage: test
41   image:
42     name: grafana/k6:latest
43     entrypoint: ['']
44   script:
45     - k6 run tests/performance/k6-load-test.js
46   rules:
47     - if: '$CI_COMMIT_BRANCH == "staging"'
48
49 # Rapport Allure
50 test-robot:allure-report:
51   stage: report
52   script:
53     - npm install -g allure-commandline
54     - allure generate tests/playwright/allure-results
55       --clean -o allure-report
56   artifacts:
57     paths:
58       - allure-report/
59     expire_in: 30 days
```

Listing B.4 – .gitlab-ci-tests.yml — Pipeline de tests (extrait)

B.5 Configuration Patrol (tests mobiles Flutter)

```
1 # Ajout dans pubspec.yaml de chaque app Flutter
2 dev_dependencies:
3   flutter_test:
4     sdk: flutter
5   integration_test:
6     sdk: flutter
7   patrol: ^3.13.0
8
9 # Execution des tests
10 # Standard (sans interactions natives) :
11 flutter test integration_test/app_test.dart
12
13 # Avec Patrol (permissions, dialogs natifs) :
14 dart pub global activate patrol_cli
15 patrol test --target integration_test/patrol_test.dart
```

Listing B.5 – Configuration Patrol dans pubspec.yaml

B.6 Couverture Angular (Istanbul/Karma)

```
1 // Dans angular.json > projects > admin-panel > architect > test
2 "options": {
3   "codeCoverage": true,
4   "codeCoverageExclude": [
5     "src/test-setup.ts",
6     "**/*.spec.ts",
7     "**/environments/**"
8   ]
9 }
10
11 // Execution : npx ng test --watch=false --browsers=
    ChromeHeadless
12 // Resultat : coverage/index.html
```

Listing B.6 – angular.json — Activation de la couverture de code